

A Major Project Report on
“FAV-ASTCL TRAFFIC
PREDICTION”

Submitted to

OSMANIA UNIVERSITY, Hyderabad

in partial fulfillment of the requirements for the award of degree

BACHELOR OF ENGINEERING
In
COMPUTER SCIENCE AND ENGINEERING

Submitted by

AHMED SAMARA (1610-22-733-077)

Under the guidance of

Mr. FAROOQ MOHIUDDIN AMJAD

Assistant Professor



DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING
NAWAB SHAH ALAM KHAN COLLEGE OF ENGINEERING &
TECHNOLOGY

Malakpet, Hyderabad

2025-2026



(A Muslim Minority Institution)

NAWAB SHAH ALAM KHAN COLLEGE OF ENGINEERING & TECHNOLOGY

Approved by AICTE,
Accredited by NAAC
Affiliated to O.U. Hyderabad. Included in 2F UGC
Recognised by Govt. of Telangana

Ref. No.

CERTIFICATE

Date _____

This is to certify that the Dissertation work entitled “**FAV-ASTCL TRAFFIC PREDICTION**” submitted by **AHMED SAMARA (1610-22-733-077)** student of the **Department of Computer Science and Engineering, Nawab Shah Alam Khan College of Engineering & Technology**, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering with Computer Science and Engineering** as a record of the bonafide work carried out by him during the academic year 2025–26.

Date of submission of thesis:

Project Supervisor

Mr. Farooq Mohiuddin Amjad

Assistant Professor

NSAKCET

Project Coordinator

Ms. Nehan Begum

Assistant Professor

NSAKCET

Head of Department

Mr. Mohammed Abdul Rawoof

Prof. And Head of the CSE Department

NSAKCET

Principal

Dr. Syed Abdul Sattar

NSAKCET

External

#16-4-1/A, New Malakpet, Hyderabad-500 024. Telangana State, INDIA.

Ph : 040-24576985, 040-65796123,

E-Mail: nsakcet@gmail.com, Website: www.nsakcet.ac.in



(A Muslim Minority Institution)

NAWAB SHAH ALAM KHAN COLLEGE OF ENGINEERING & TECHNOLOGY

Approved by AICTE,

Accredited by NAAC

Affiliated to O.U. Hyderabad. Included in 2F UGC

Recognised by Govt. of Telangana

DECLARATION

I hereby declare that the project work entitled “**FAV-ASTCL TRAFFIC PREDICTION**” submitted to the Department of **Computer Science and Engineering of NAWAB SHAH ALAM KHAN COLLEGE OF ENGINEERING AND TECHNOLOGY**, affiliated to Osmania University, Hyderabad in the partial fulfillment of the requirement for award of the degree in **BACHELOR OF ENGINEERING in CSE** is a bonafide work done by the undersigned.

AHMED SAMARA

(1610-22-733-077)

ACKNOWLEDGEMENT

The satisfaction that accomplished completion of any task would be incomplete without the mention of people who made it possible and whose encouragement and guidance has been a source of inspiration through the source of the project.

I express our profound sense of gratitude to **Dr. Syed Abdul Sattar, Principal** for his constant support.

I would like to express our sincere thank to **Dr. Mohammad Sanaullah Qaseem, Professor, CSE department and Dean of Computer Studies** for sharing his experience and valuable knowledge to enhance our real time learning experience.

I would like to express our sincere thanks to **Mr. Mohammed Abdul Rawoof, Associate Professor & HOD, CSE Department** and our Internal Guide **Mr Farooq Mohiuddin Amjad, Assistant Professor**, for their earnest efforts and timely suggestion and that motivated me to come out with satisfactory project work.

I thank everybody who directly or indirectly played a vital role in finishing my project work with less difficulty.

AHMED SAMARA

(1610-22-733-077)

ABSTRACT

Urban traffic congestion remains one of the most critical challenges facing modern smart city infrastructure. Existing deep learning approaches rely on static road graphs and overlook exogenous factors such as weather conditions, time-of-day patterns, and special events, leading to degraded performance under real-world conditions.

This thesis introduces FAV-ASTCL (Forecasting-Aware Versatile Adaptive Spatio-Temporal Context Learning), a novel deep learning framework for real-time urban traffic speed forecasting. The architecture addresses these limitations through three key innovations: a Learnable Context Selector that dynamically re-wires the spatial graph at every inference step, an Exogenous Gating Mechanism that filters weather and temporal signals to prevent over-reaction to irrelevant environmental changes, and a Residual Online Adapter that corrects sudden prediction spikes typically missed by standard GRU encoders.

FAV-ASTCL is trained on the METR-LA benchmark dataset comprising 207 road sensors across Los Angeles, and evaluated on five major traffic hubs in Hyderabad, India. The model generates multi-horizon speed predictions at 5, 10, and 15-minute intervals using a 60-minute historical window and six exogenous context features. FAV-ASTCL achieves a MAE of 3.633 km/h, RMSE of 8.842, and MAPE of 8.22%, corresponding to an overall accuracy of 91.78%, an improvement of over 15 percentage points compared to the ASTCL baseline. A complete deployment is also implemented, featuring a Flask REST API, real-time browser dashboard, and integration with the TomTom Traffic API and WeatherAPI.

Table of contents

1. INTRODUCTION	1
1.1 Background of the Study	1
1.2 Problem Statement	2
1.3 Motivation	3
1.4 Objectives	5
1.5 Scope of the Project	5
2. Literature Survey	7
2.1 Traditional Traffic Forecasting Methods	7
2.2 Machine Learning Approaches	8
2.3 Recurrent Neural Networks for Traffic	8
2.4 Graph Neural Networks for Spatio-Temporal Forecasting	9
2.5 Context-Aware and Exogenous-Informed Models	10
2.6 Adaptive Graph Mechanisms	11
2.7 Residual Learning and Online Adaptation	12
2.8 Summary and Research Gap	13
3. System Analysis.....	15
3.1 Software Requirements Specification	15
3.2 Datasets	15
3.2.1 METR-LA Traffic Dataset	15
3.2.2 Exogenous Context	16
3.3 Data Preprocessing	17
3.3.1 Normalisation	17
3.3.2 Sliding Window Construction	18
3.4 Model Architecture	18
3.4.1 Learnable Context Selector	19
3.4.2 Exogenous Encoder and Gating	20
3.4.3 Concatenation and Temporal Modelling	21
3.4.4 Residual Online Adapter	21

3.4.6 Parameter Summary	21
3.5 Training Procedure	22
3.6 Inference Pipeline for Live Deployment	24
4. Model and Backend Implementation	25
4.1 Software Stack and Environment	25
4.2 System Architecture	27
4.3 Model Implementation	29
4.3.1 Exogenous Encoder	29
4.3.2 Learnable Context Selector	30
4.3.3 Full Forward Pass	31
4.4 Flask REST API	33
4.4.1 Sensor Configuration	33
4.4.3 API Response Format	35
4.5 External API Integration	36
4.5.1 TomTom Traffic API	37
4.5.2 Weather API	37
4.5.3 HERE Maps Traffic Client	38
5. Results and Performance Analysis.....	39
5.1 Experimental Setup	39
5.2 Training Results	41
5.3 Evaluation Metrics	42
5.4 Performance Comparison	43
5.5 Sensor-Level Forecast Analysis	45
6: Frontend and Visualisation System	47
6.1 UML Diagrams	47
6.1.1 Use Case Diagram	48
6.1.2 Class Diagram	49
6.1.3 Sequence Diagram	50
6.1.4 Activity Diagram	51
6.2 Frontend Dashboard	52

6.2.1 Page Layout	53
6.2.2 CSS Design System	54
6.2.3 JavaScript Data Flow	55
6.3 Sensor-Level Prediction View	56
6.4 Console Streaming Mode	57
6.5 Streamlit Dashboard (Alternative UI)	58
7. Future Scope	60
7.1 Real-Time Smart City Deployment	60
7.2 Advanced Model Enhancements	62
7.2.1 Improved Dynamic Graph Learning	62
7.2.2 Advanced Deep Learning Architectures	63
7.3 IoT and Cloud Integration	65
7.4 Mobile and Web Platform Expansion	66
7.5 Intelligent Traffic Management Systems	67
8. Conclusion	69
8.1 Project Summary	69
8.2 Objectives Achieved	70
8.3 Key Contributions	71
8.4 System Performance Overview	72
8.5 Final Remarks	73
References	74

List of Tables

Table: 2.8 Comparison of Traffic Forecasting Methods	13
Table: 3.2.1 METR-LA Dataset Statistics	16
Table: 3.2.2 PEMS-BAY Dataset Statistics	17
Table: 3.4.6 Parameter Summary	22
Table: 4.1 Software Stack	27
Table: 4.2 Sensor Configuration	34
Table: 4.3 Flask API Endpoints	34
Table: 5.1 Experimental Configuration	41
Table 5.2: Training MAE Progression	42
Table 5.3: Performance Comparison Between ASTCL and FAV-ASTC	44
Table 5.4: Illustrative Sensor-Level Forecast Summary	45

List of Figures

Fig: 3.4 Model Architecture	18
Fig: 3.5 Training Procedure.....	23
Fig: 4.2 System Architecture.....	27
Fig: 6.1 Use Case Diagram.....	48
Fig: 6.2 Class Diagram.....	49
Fig: 6.3 Sequence Diagram.....	50
Fig: 6.4 Activity Diagram.....	51
Fig: 6.5 Frontend Dashboard.....	52
Fig: 6.6 Sensor-Level Prediction View.....	57

CHAPTER 1

1. INTRODUCTION

The twenty-first century has witnessed an unprecedented acceleration in global urbanisation. As millions of people migrate from rural areas to cities in search of better economic opportunities and improved living standards, urban centres are experiencing a surge in population density that places enormous strain on existing infrastructure. Among the many systems affected by this growth, road transportation networks stand out as particularly vulnerable. The sheer volume of vehicles competing for limited road space during peak hours has made traffic congestion a near-universal phenomenon in major cities across the world. Beyond the daily inconvenience experienced by commuters, congestion carries significant economic, environmental, and social consequences, including wasted fuel, increased greenhouse gas emissions, delayed emergency response times, and reduced overall productivity.

Addressing these challenges at scale requires a fundamental shift in how cities manage and anticipate traffic. Reactive approaches, where congestion is responded to only after it has already formed, are no longer sufficient. What modern cities require are proactive, data-driven systems capable of forecasting traffic conditions minutes or hours in advance, allowing traffic management authorities and commuters alike to make informed decisions before bottlenecks develop. This is the domain of intelligent traffic forecasting, and it is here that deep learning has emerged as a transformative force. This project presents FAV-ASTCL (Forecasting-Aware Versatile Adaptive Spatio-Temporal Context Learning), a novel deep learning framework designed to push the boundaries of what is achievable in real-time urban traffic speed forecasting by overcoming the critical limitations that constrain existing state-of-the-art models.

1.1 Background of the Study

The rapid urbanisation of modern cities has led to an unprecedented increase in the volume of road traffic, transforming congestion from an occasional inconvenience into a chronic infrastructure problem. Cities like Hyderabad, India, serve as particularly illustrative examples of this phenomenon. As one of India's fastest-growing technology hubs, Hyderabad hosts a dense IT corridor spanning key areas such as Hitech City, Gachibowli, Punjagutta, Secunderabad, and LB Nagar. During morning and evening peak hours, these corridors experience intense bidirectional traffic flows driven by the daily movement of hundreds of thousands of IT professionals, students, and commuters. The spatial concentration of workplaces and residential areas, combined with road networks that have not expanded proportionally to population growth, creates a highly volatile and difficult-to-predict traffic environment.

Traffic congestion in such environments does not merely cause inconvenience. According to various urban mobility studies, congestion imposes measurable economic costs by extending commute times, increasing fuel consumption, and reducing the effective productivity of both individuals and logistics networks. For emergency services, congested roads can mean the difference between timely and delayed intervention. The cumulative societal cost of unmanaged traffic congestion in large Indian cities runs into billions of rupees annually, underscoring the urgency of developing reliable and accurate forecasting tools.

Traffic forecasting, broadly defined as the task of predicting future traffic speed, flow, or occupancy from historical observations, has been a subject of academic and industrial research for several decades. The earliest approaches to this problem were rooted in classical statistical time-series analysis. Models such as ARIMA (Autoregressive Integrated Moving Average) and its seasonal variant, SARIMA, gained widespread use in the 1980s and 1990s due to their mathematical tractability and interpretability. These models treat each sensor or road segment as an independent time series, applying autoregressive and moving-average components to extrapolate future values from past observations. While effective for capturing short-term linear trends under stable conditions, ARIMA-type models have well-documented limitations when applied to the inherently non-linear and spatially coupled dynamics of urban road networks. They cannot model the influence that congestion at one location has on traffic conditions at a geographically connected location, which is a fundamental feature of real-world traffic propagation.

1.2 Problem Statement

Urban traffic forecasting is a foundational component of intelligent transportation systems, underpinning applications that range from dynamic route guidance and signal timing optimisation to freight logistics planning and emergency vehicle dispatch. At its core, the problem is one of structured sequence prediction: given a history of traffic observations across a network of sensors, the task is to produce accurate estimates of future traffic conditions at each sensor location across multiple future time horizons.

$$\mathbf{G} = (\mathbf{V}, \mathbf{E})$$

In this work, the traffic network is formally modelled as a graph where $\mathbf{V}$ represents the set of N sensor nodes distributed across different road segments and intersections, and $\mathbf{E}$ denotes the set of edges encoding the connectivity or spatial proximity between these nodes. Each node captures traffic-related measurements such as vehicle speed or flow at regular intervals, enabling the road

network to be represented as a dynamic spatio-temporal system in which both location and time are first-class dimensions of the data.

$$\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\}$$

denote the sequence of historical traffic observations of dimension $RT \times N \times 1$, where T is the number of historical time steps and N is the total number of sensor nodes. These observations encode the temporal evolution of traffic conditions across the entire network and form the primary input to the forecasting model. In addition to raw traffic measurements, external contextual factors play a significant role in shaping traffic dynamics. These exogenous features are represented as $\mathbf{Z} = \{\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_T\}$ of dimension $RT \times 6$, where each feature vector includes six variables: temperature, humidity, wind speed, hour of day, day of week, and a binary weekend or holiday indicator. These features collectively capture both the meteorological environment and the temporal context in which traffic observations are made.

The prediction objective is to forecast the traffic state across all N sensor nodes simultaneously for multiple future time horizons. Formally, the target output is expressed as: $\hat{\mathbf{Y}} = \{\hat{\mathbf{y}}_{T+1}, \hat{\mathbf{y}}_{T+2}, \hat{\mathbf{y}}_{T+3}\}$ where each $\hat{\mathbf{y}}_{T+k}$ represents the predicted traffic state across all sensors at future time step $T+k$. In this project, the three prediction horizons correspond to short-term forecasts of 5, 10, and 15 minutes into the future, respectively.

$$\hat{\mathbf{Y}} = \{\hat{\mathbf{y}}_{T+1}, \hat{\mathbf{y}}_{T+2}, \hat{\mathbf{y}}_{T+3}\} \in \mathbb{R}^{N \times 3}$$

The central challenge in solving this prediction problem lies in accurately modelling the dual dependency structure of traffic data. On one hand, spatial dependencies capture the influence that traffic conditions at one location have on nearby or connected locations, a relationship that is mediated by the graph structure of the road network. Compounding these challenges, traffic patterns can change abruptly and unpredictably due to external disturbances such as heavy rainfall, road accidents, large-scale public events, or sudden changes in signal timings. These non-stationary disruptions make the forecasting task highly non-linear and demand a model that can dynamically adapt to shifting conditions rather than relying on fixed learned representations.

1.3 Motivation

The motivation for this project arises directly from the gap between what existing traffic forecasting models offer and what real-world deployment scenarios actually require. Accurate traffic forecasting is indispensable for the functioning of modern intelligent transportation systems, yet the most widely used deep learning models are built on assumptions that do not hold in practice.

Traditional traffic prediction models, including many deep learning baselines, are designed under the implicit assumption that traffic patterns are primarily a function of historical traffic data alone. While historical observations are undeniably the strongest predictor of near-future traffic conditions, they are far from the only relevant signal. Real-world traffic behaviour is shaped continuously by a web of interacting factors that extend well beyond past speed readings. Weather conditions constitute one of the most significant of these factors. Rainfall reduces vehicle speeds directly by degrading road traction and visibility, and indirectly by altering driver behaviour toward greater caution and slower acceleration. Temperature extremes affect vehicle performance and, in some cases, road surface integrity. Humidity levels, particularly when combined with temperature, influence pedestrian behaviour at crossings and the frequency of minor incidents. None of these effects are captured by models that operate exclusively on traffic speed time series.

Temporal context presents a similarly important but underutilised source of information. The traffic profile of a weekday morning in an IT-heavy corridor like Gachibowli is fundamentally different from that of a Sunday afternoon or a national holiday. While models can in principle learn these patterns from sufficiently long historical sequences, explicitly encoding hour-of-day, day-of-week, and holiday indicators as input features allows the model to generalise more robustly to novel combinations of conditions and reduces the amount of historical data required to learn stable temporal patterns.

Existing models such as the ASTCL (Adaptive Spatio-Temporal Context Learning) framework have demonstrated impressive performance on standard benchmarks by effectively capturing spatial and temporal dependencies within traffic networks. However, the absence of exogenous information in these models limits their ability to generalise under conditions that deviate from the historical norm. A model that has never been exposed to the correlation between rainfall and reduced traffic speed will inevitably under-perform when it encounters a monsoon-induced slowdown during inference. Similarly, a model with a static spatial graph will fail to adapt when the effective connectivity of the road network is altered by an unexpected event.

These limitations are not merely theoretical. In the context of Hyderabad, a city that experiences intense monsoon seasons and large-scale public events that significantly disrupt normal traffic flow, the failure of static and context-blind models has direct practical consequences. It is precisely this gap between benchmark performance and real-world reliability that motivates the development of FAV-ASTCL. By integrating weather-based exogenous features through a principled gating mechanism, adopting a dynamic and learnable graph structure, and introducing a residual correction layer to handle sudden prediction errors, FAV-ASTCL is designed to deliver the

kind of robust, context-aware forecasting that practical deployment in a city like Hyderabad demands.

1.4 Objectives

This project is guided by a set of clearly defined technical objectives, each of which addresses a specific limitation of existing approaches or a practical requirement for real-world deployment.

The primary objective is to design and implement a deep learning architecture that dynamically adapts its spatial graph representation at every inference step based on prevailing traffic conditions, thereby moving beyond the static adjacency matrices that constrain conventional spatio-temporal models. Closely related to this is the objective of incorporating exogenous signals, specifically weather variables and temporal indicators, into the forecasting pipeline through a learnable gating mechanism. This mechanism is designed to selectively amplify context features that are genuinely relevant to current traffic conditions while suppressing those that would introduce noise or instability into the predictions.

On the empirical side, the project aims to train and rigorously evaluate the proposed model on the widely used METR-LA benchmark dataset, ensuring that its performance can be compared fairly against established baselines under standardised conditions. Beyond benchmark evaluation, the project also aims to demonstrate the practical deployability of the system by implementing a complete end-to-end web service consisting of a Flask REST API server and a live browser dashboard providing real-time traffic speed forecasts for five key locations in Hyderabad. The final objective is to demonstrate a clear and statistically meaningful improvement over the ASTCL baseline across all three standard evaluation metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE).

1.5 Scope of the Project

This project encompasses the full lifecycle of a production-oriented deep learning system, from raw data acquisition and preprocessing through model design, training, evaluation, and live deployment.

On the data side, the scope includes the acquisition, normalisation, and structured windowing of historical traffic data from the METR-LA dataset as well as synthetic exogenous context data representative of real-world conditions in Hyderabad. The preprocessing pipeline transforms raw sensor readings and contextual features into properly formatted sliding-window training samples suitable for multi-step forecasting.

The architectural scope covers the complete design and implementation of all three novel components that distinguish FAV-ASTCL from prior work: the Learnable Context Selector, which drives dynamic graph re-wiring at inference time; the Exogenous Gating Mechanism, which controls the influence of weather and temporal features; and the Residual Online Adapter, which provides post-hoc correction of large prediction errors. The training scope includes end-to-end optimisation of the full model using the Adam optimiser with a Mean Absolute Error loss function on the METR-LA training split, followed by systematic hyperparameter tuning and evaluation on the held-out test split.

The deployment scope extends the project into the domain of practical software engineering, covering the implementation of a Flask-based REST API that serves real-time speed forecasts and a JavaScript-driven browser dashboard that visualises live sensor states, congestion levels, and multi-horizon forecast trajectories for five Hyderabad sensor hubs. The project also includes optional live data integration through external APIs, namely the TomTom Traffic API for real-time speed readings and WeatherAPI for current meteorological conditions, enabling the system to operate as a fully live forecasting service rather than a purely offline evaluation framework.

CHAPTER 2

2. LITERATURE SURVEY

The field of traffic forecasting has evolved substantially over the past four decades, progressing from simple statistical models rooted in time-series analysis to sophisticated deep learning architectures that jointly model spatial and temporal dynamics across entire road networks. This chapter traces that evolution, reviewing the key methodological milestones and the insights they contributed, while also identifying the persistent limitations that have motivated the development of FAV-ASTCL.

2.1 Traditional Traffic Forecasting Methods

The earliest systematic approaches to traffic forecasting were grounded in classical statistical time-series analysis. The Autoregressive Integrated Moving Average (ARIMA) model and its variants were among the first tools applied to this problem, gaining widespread use from the 1970s onwards due to their mathematical tractability and relative ease of implementation. ARIMA models capture linear temporal dependencies in a single time series by combining autoregressive terms, which relate current values to past observations, with moving average terms that account for past forecast errors. Under stable and low-variance conditions, these models can produce reasonable short-horizon predictions for individual road sensors.

However, the limitations of ARIMA become apparent quickly in the context of real-world urban traffic. The model treats each sensor as an independent time series, entirely ignoring the spatial dependencies that exist between road segments in a connected network. Traffic congestion does not develop in isolation; a bottleneck at one junction propagates its effects upstream and downstream across the network, and no univariate statistical model can represent this propagation. To address some of these shortcomings, several extensions were proposed. Seasonal ARIMA (SARIMA) introduced the ability to model periodic patterns, making it better suited to capturing daily and weekly traffic cycles. Vector Autoregression (VAR) extended the framework to multivariate time series, allowing interactions between multiple sensors to be considered. Despite these improvements, VAR models scale poorly to large networks due to the quadratic growth in parameters with the number of variables, and they remain fundamentally limited in their ability to capture non-linear dynamics.

Another widely studied classical approach is the Kalman filter, which offers a principled framework for sequential state estimation under Gaussian noise assumptions. Kalman filters can incorporate new observations dynamically, making them attractive for online prediction tasks where sensor readings arrive in a stream. However, their reliance on linear system dynamics and carefully

tuned noise covariance matrices makes them poorly suited to the highly non-linear and stochastic nature of urban traffic. The requirement for precise system modelling restricts their scalability and generalisation, particularly in large and complex road networks. Overall, while classical statistical methods established the foundational vocabulary of traffic forecasting and provided useful baselines, their inability to model non-linearity, spatial correlations, or dynamic behaviour ultimately limited their performance in modern intelligent transportation contexts.

2.2 Machine Learning Approaches

The growing availability of large traffic datasets and advances in computational power during the 2000s prompted researchers to explore machine learning methods that could capture the non-linear relationships in traffic data more effectively than statistical baselines. Support Vector Regression (SVR) was among the first machine learning techniques to demonstrate meaningful improvements over ARIMA on traffic prediction benchmarks. SVR maps input features into a high-dimensional space and fits a regression hyperplane that maximises the margin between predictions and true values, enabling it to model moderately complex non-linear functions.

Ensemble methods such as Random Forests and gradient-boosted trees further improved predictive accuracy by combining the outputs of multiple weak learners trained on different subsets of the data. These approaches are robust to outliers and overfitting and demonstrated competitive performance on several short-horizon forecasting tasks. Artificial Neural Networks (ANNs), in the form of standard feedforward or multilayer perceptron architectures, were also applied to traffic prediction during this period. Their ability to approximate arbitrary non-linear functions made them more expressive than SVR, and they served as important stepping stones toward the deep learning models that followed.

Nevertheless, a critical limitation pervaded all of these approaches: they treated sensors as independent entities and required manual feature engineering to incorporate any notion of spatial structure. Neighbouring sensors' readings had to be explicitly included as input features, and there was no principled way to model the relational structure of a road network. As a result, these models could not scale gracefully to large multi-sensor networks, and their performance degraded when spatial dependencies were the primary driver of traffic dynamics. This fundamental gap set the stage for the adoption of recurrent neural networks.

2.3 Recurrent Neural Networks for Traffic

The introduction of recurrent neural network architectures capable of learning long-range temporal dependencies marked a significant turning point in traffic forecasting. Long Short-Term

Memory (LSTM) networks, proposed by Hochreiter and Schmidhuber, addressed the vanishing gradient problem that had long plagued standard RNNs through the use of three gating mechanisms: an input gate, a forget gate, and an output gate. These gates collectively control the flow of information into, out of, and within a memory cell, allowing the network to retain relevant information across arbitrarily long sequences. Gated Recurrent Units (GRUs), a later and computationally more efficient variant, achieved comparable performance using only two gates, the reset gate and the update gate, by merging the cell and hidden state into a single representation.

Both LSTM and GRU architectures proved highly effective at capturing the temporal autocorrelation structures present in traffic time series, including the regular daily and weekly peaks and the short-term dependencies between consecutive time steps. Several studies applied stacked LSTM architectures to multi-sensor traffic prediction by concatenating the readings of spatially adjacent sensors as additional input features before temporal modelling, effectively incorporating a rudimentary form of spatial context. While these approaches improved over purely univariate models, they suffered from two significant drawbacks. First, the spatial feature engineering was manual and static, requiring domain expertise to determine which neighbouring sensors to include and how to weight them. Second, the concatenation approach does not capture the relational structure of the road network in any principled way and scales poorly to large networks with hundreds of sensors.

These limitations made it clear that the spatial dimension of traffic forecasting required a more structured and learnable representation, one that could be derived automatically from the connectivity of the road network itself. This realisation drove the adoption of graph-based deep learning architectures, which would ultimately define the state of the art in the field.

2.4 Graph Neural Networks for Spatio-Temporal Forecasting

The modelling of road networks as graphs, where nodes represent sensors and edges encode road connectivity or physical proximity between sensors, proved to be a natural and powerful representation that enabled a step change in traffic forecasting performance. Graph Neural Networks (GNNs) operate directly on this graph structure, propagating information between nodes according to the topology of the road network and learning representations that capture both local and global spatial dependencies.

The Diffusion Convolutional Recurrent Neural Network (DCRNN) was among the first architectures to combine graph-based spatial modelling with recurrent temporal modelling in a unified framework. DCRNN models traffic propagation as a diffusion process on a directed graph,

using bidirectional random walks to capture both upstream and downstream traffic influence. A sequence-to-sequence architecture with GRU-based encoder and decoder components handles the temporal dimension. DCRNN produced state-of-the-art results on the METR-LA benchmark at the time of its publication and established graph diffusion as a powerful spatial operator for traffic data. However, its adjacency matrix is fixed and pre-computed from sensor distances, meaning the model cannot adapt to changes in effective network connectivity during inference.

The Spatio-Temporal Graph Convolutional Network (STGCN) took a different approach, replacing the recurrent components with gated one-dimensional convolutions along the time axis, combined with spectral graph convolutions for spatial modelling. This fully convolutional architecture is significantly faster to train than DCRNN due to the parallelisability of convolutions compared to sequential recurrence, and it achieved competitive accuracy while reducing training time substantially. However, STGCN retains the static graph assumption of its predecessors, limiting its adaptability to dynamic network conditions.

Graph WaveNet introduced the important concept of a learnable adaptive adjacency matrix, constructed as the product of two randomly initialised embedding matrices that are updated during training through gradient descent. This allowed the model to discover spatial dependencies that extend beyond physical road proximity, capturing latent relationships between sensors that may be geographically distant but functionally correlated. The temporal component of Graph WaveNet employed dilated causal convolutions, which provide efficient multi-scale temporal modelling by exponentially increasing the receptive field across layers. While the adaptive adjacency matrix in Graph WaveNet is a significant advance, it is still a fixed learned parameter at inference time and does not change based on current traffic conditions.

AGCRN (Adaptive Graph Convolutional Recurrent Network) extended this direction by combining node-specific parameter learning with adaptive graph generation, allowing different nodes to learn distinct transformation parameters while still sharing a common graph structure. This architecture achieved strong performance on multiple benchmarks while also reducing overall parameter count relative to comparable models. Despite these advances, AGCRN's graph structure remains static during inference, and none of the GNN-based models reviewed so far incorporate any form of exogenous context.

2.5 Context-Aware and Exogenous-Informed Models

A growing body of research has recognised that traffic dynamics are not fully determined by past traffic observations alone and that external contextual signals carry substantial predictive value.

This has motivated a line of work focused on incorporating temporal and environmental context into deep learning traffic models.

STAWnet introduced a temporal attention mechanism that explicitly attends across different time slots in the prediction window, demonstrating that traffic patterns during holidays and weekends differ significantly from those on regular weekdays and that models which do not account for this distinction systematically underperform during atypical periods. GMAN (Graph Multi-Attention Network) adopted a transformer-inspired architecture with multi-head attention operating across both the spatial and temporal dimensions, encoding spatial and temporal embeddings that allow the model to attend to different parts of the network at different times. While GMAN represents a meaningful advance in representational expressiveness, it does not incorporate real meteorological data and its spatial attention mechanism is still fundamentally tied to fixed node embeddings derived from road network structure.

External research has quantified the impact of weather on traffic more directly. Studies have shown that rain events can reduce average traffic speeds by ten to twenty percent on urban arterials, and that models which ignore precipitation data underperform significantly during adverse weather conditions. Temperature extremes and high humidity levels also correlate with measurable changes in driving behaviour and incident rates. Despite this well-documented evidence, the majority of deep learning traffic models continue to be trained and evaluated on traffic speed data alone, treating weather as an implicit nuisance variable rather than an explicit and learnable input feature. This omission represents a clear gap between the capabilities of existing models and the requirements of robust real-world deployment.

2.6 Adaptive Graph Mechanisms

The idea of dynamically computing graph structure from the data itself, rather than fixing it from physical road distances, is central to the design of FAV-ASTCL and has been explored from several different angles in the recent literature. GTS (Graph for Time Series) proposed learning a probabilistic graph structure jointly with the forecasting model by computing pairwise node similarity from input representations and treating the resulting graph as a latent variable. NRI (Neural Relational Inference) applied a variational encoder-decoder framework to infer the relational structure among interacting agents from trajectory data, using discrete latent variables to represent different types of interaction.

These approaches represent meaningful progress toward data-driven graph construction, but they typically learn a fixed or slowly adapting graph structure that does not change at inference time

based on current observations. The key distinction in FAV-ASTCL is that the dynamic adjacency matrix is recomputed from the actual current input at every single inference step, rather than being stored as a learned parameter. This means the spatial graph re-wires itself every five minutes based on what is genuinely happening across the sensor network at that moment, enabling the model to respond in real time to sudden disruptions such as accidents, road closures, or rainfall events that instantaneously alter which parts of the network are influencing each other.

In traditional graph-based traffic forecasting models, the predefined adjacency matrix captures only static structural relationships and cannot reflect the fact that, during peak congestion, functionally distant sensors may become more predictively relevant to each other than physically adjacent ones. The adaptive mechanism in FAV-ASTCL overcomes this limitation by measuring similarity between node representations derived from the current traffic window, constructing a soft adjacency matrix that reflects the current state of the network rather than its average historical structure. This approach also reduces the reliance on hand-crafted domain knowledge about road topology, making the framework more transferable across different city networks.

2.7 Residual Learning and Online Adaptation

Residual connections, introduced by He et al. in the ResNet architecture for image classification, have since become one of the most widely adopted structural elements in deep learning. The key insight behind residual learning is that it is often easier for a network layer to learn a small corrective adjustment, or residual, on top of an identity mapping than to learn the full target transformation from scratch. This formulation improves gradient flow during backpropagation, reduces the risk of vanishing gradients in deep networks, and empirically accelerates convergence. Residual connections have since been adopted across domains ranging from natural language processing to time-series forecasting.

In the context of traffic forecasting, residual learning is particularly valuable for handling the short-term fluctuations and abrupt transient events that standard recurrent models tend to smooth over. GRU encoders, by their nature, produce predictions that reflect a weighted integration of past observations across the entire input window. This smoothing behaviour is generally beneficial for stable conditions but introduces systematic lag when traffic conditions change abruptly, such as during the sudden onset of congestion caused by an accident or a traffic signal failure. The residual MLP adapter in FAV-ASTCL directly addresses this limitation by applying a lightweight corrective transformation to the GRU hidden state, learning to predict and subtract the systematic bias that arises under these transient conditions.

Beyond improving prediction accuracy during sudden events, the residual adapter also contributes to more stable training dynamics by providing an additional gradient pathway through the network. This is particularly relevant in the FAV-ASTCL architecture, where the main encoder must simultaneously learn to handle dynamic graph structures and exogenous gating, increasing the complexity of the optimisation landscape. The residual adapter acts as a complementary learner that can specialise in the high-frequency residual signal while the main encoder focuses on the broader spatio-temporal patterns, resulting in a more effective division of representational labour.

2.8 Summary and Research Gap

The preceding review illustrates a clear trajectory of progress in traffic forecasting, from univariate statistical models through spatial deep learning architectures and toward context-aware and adaptive systems. Each generation of methods addressed specific limitations of its predecessors while introducing new capabilities, yet no single existing approach simultaneously addresses all three of the key challenges identified in this work.

The table below summarises the capabilities of the key methods reviewed across the four critical dimensions of spatial modelling, temporal modelling, adaptive graph construction, and exogenous context integration.

Method	Spatial Modelling	Temporal Modelling	Adaptive Graph	Exogenous Context
ARIMA	None	Statistical	No	No
LSTM / GRU	None	Recurrent	No	No
DCRNN	Graph Diffusion	GRU	No	No
STGCN	Spectral GCN	1D Conv	No	No
Graph WaveNet	Adaptive GCN	Dilated Conv	Yes (fixed)	No
AGCRN	Adaptive GCN	GRU	Yes (fixed)	No
GMAN	Multi-head Attention	Attention	No	No
FAV-ASTCL (Ours)	Dynamic Similarity	GRU + Adapter	Yes (per step)	Yes (Gated)

Table: 2.8 Comparison of Traffic Forecasting Methods

As the table makes clear, no existing method simultaneously employs input-driven dynamic graph re-wiring at every inference step, gated exogenous context fusion, and a residual online

adapter for transient error correction. Models such as Graph WaveNet and AGCRN adopt adaptive graph mechanisms but fix the learned graph structure at inference time and include no external context. Context-aware models such as STAWnet and GMAN improve temporal representation but do not incorporate real weather features and do not dynamically re-wire the spatial graph. The inability to respond effectively to sudden and transient changes in traffic conditions, such as accidents, rainfall, or large public events, remains a gap shared by virtually all existing architectures. FAV-ASTCL is specifically designed to fill this gap by integrating all three complementary capabilities within a single unified framework, and the remainder of this report demonstrates its effectiveness both on a standard benchmark and in a real deployment scenario.

CHAPTER 3

3. SYSTEM ANALYSIS

This chapter presents a comprehensive account of the system design, data infrastructure, and architectural components that together constitute the FAV-ASTCL framework. It covers the software requirements of the system, the datasets used for training and evaluation, the preprocessing pipeline applied to raw sensor data, the detailed architecture of each novel model component, the training procedure, and the inference pipeline designed for live deployment. Together, these elements define a complete and production-oriented deep learning system for real-time urban traffic speed forecasting.

3.1 Software Requirements Specification

The FAV-ASTCL system is implemented entirely in Python and relies on a carefully selected stack of open-source libraries that together provide the numerical computation, deep learning, web serving, and data visualisation capabilities required across all phases of the project. The core deep learning framework is PyTorch, chosen for its dynamic computation graph, which facilitates the implementation of the input-dependent dynamic adjacency matrix that lies at the heart of the FAV-ASTCL architecture. NumPy and Pandas are used extensively throughout the data preprocessing pipeline for array manipulation, windowing, and feature engineering. The web deployment layer is built on Flask, a lightweight Python web framework that provides the REST API server through which the trained model is served to end users. The browser-based dashboard is implemented using HTML, CSS, and vanilla JavaScript, with Chart.js handling the real-time visualisation of speed forecasts. External data integration is achieved through the TomTom Traffic API and WeatherAPI, both accessed via standard HTTP requests from within the Flask server.

3.2 Datasets

3.2.1 METR-LA Traffic Dataset

The primary training and evaluation dataset used in this project is METR-LA, a widely used benchmark in the traffic forecasting literature. METR-LA contains speed readings collected from 207 loop detectors deployed on the highway network of Los Angeles, California. Measurements are recorded at five-minute intervals, yielding a time series that spans approximately four months from March to June 2012 and contains over six million individual sensor readings. The dataset is spatially structured through a 207 by 207 adjacency matrix that encodes the physical distances between sensor pairs, with a Gaussian kernel applied to convert distances into edge weights. This

adjacency matrix serves as the static graph component within the FAV-ASTCL architecture, complementing the dynamic graph computed at each inference step.

The choice of METR-LA as the primary benchmark is motivated by its widespread use in the traffic forecasting literature, which enables direct and fair comparison of FAV-ASTCL against established baselines such as DCRNN, STGCN, Graph WaveNet, and AGCRN. The dataset captures a broad range of traffic conditions including morning and evening peak hours, weekend lulls, and occasional disruption events, providing a sufficiently diverse training distribution for the model to learn robust spatio-temporal representations.

Property	Value
Number of Sensors	207
Location	Los Angeles, California, USA
Time Period	March 2012 – June 2012
Total Duration	~4 months
Sampling Interval	5 minutes
Total Time Steps	34,272
Feature Recorded	Traffic Speed (km/h)
Mean Speed	34.5 km/h
Standard Deviation	12.8 km/h
Missing Data Rate	~8%
Train / Test Split	80% / 20%
Adjacency Matrix	207×207 (Gaussian kernel on road distance)

Table: 3.2.1 METR-LA Dataset Statistics

3.2.2 Exogenous Context

In addition to the core traffic speed data, the model is supplied with an exogenous context array stored in the file `exo_hyderabad.npy`. This array contains six contextual features recorded at each time step, constructed to reflect realistic conditions in Hyderabad, India: temperature in degrees Celsius, relative humidity as a percentage, wind speed in kilometres per hour, hour of day

as an integer from zero to twenty-three, day of week as an integer from zero to six, and a binary flag indicating whether the current time step falls on a weekend or public holiday. Although this array is synthetically constructed for the purposes of this project, it is designed to replicate the statistical properties and seasonal patterns of Hyderabad's climate and urban calendar, providing a realistic proxy for the kind of live exogenous data that would be sourced from WeatherAPI and a calendar utility in a fully live deployment.

Property	Value
Number of Sensors	325
Location	Bay Area, California, USA
Time Period	January 2017 – June 2017
Total Duration	~6 months
Sampling Interval	5 minutes
Total Time Steps	52,116
Feature Recorded	Traffic Speed (km/h)
Mean Speed	62.4 km/h
Standard Deviation	17.1 km/h
Missing Data Rate	~0.003%
Train / Test Split	70% / 10% / 20% (Train / Val / Test)
Adjacency Matrix	325×325 (Gaussian kernel on road distance)

Table: 3.2.2 PEMS-BAY Dataset Statistics

3.3 Data Preprocessing

3.3.1 Normalisation

Before being passed into the model, all traffic speed values are normalised using a Z-score transformation computed from the training split of the METR-LA dataset. Specifically, each speed value v is transformed as $\mathbf{x}_{\text{norm}} = \frac{v - \mu}{\sigma}$, where $\mu = 34.5$ km/h and $\sigma = 12.8$ km/h represent the mean and standard deviation of training speeds respectively. This normalisation step ensures that

the input values fall within a numerically stable range for gradient-based optimisation and prevents any single sensor with a high absolute speed from dominating the learning signal. The exogenous features are normalised independently using their own per-feature means and standard deviations, ensuring that each of the six context variables contributes on an equal footing to the exogenous encoder regardless of its original unit or scale.

3.3.2 Sliding Window Construction

The normalised traffic and exogenous data are transformed into fixed-length input-output pairs using a sliding window procedure. Each training sample consists of a traffic input window of twelve consecutive time steps, corresponding to sixty minutes of historical observations, and an exogenous context window of the same length. The target output for each sample is the traffic speed across all sensor nodes at the three subsequent time steps, corresponding to five, ten, and fifteen minutes into the future. The window slides forward by one time step between successive samples, maximising the number of training examples while ensuring that the model is exposed to all possible temporal positions within the dataset. An 80/20 chronological split is applied to divide the full sequence into training and test sets, with no shuffling across the split boundary to prevent information leakage from future observations into the training set.

3.4 Model Architecture

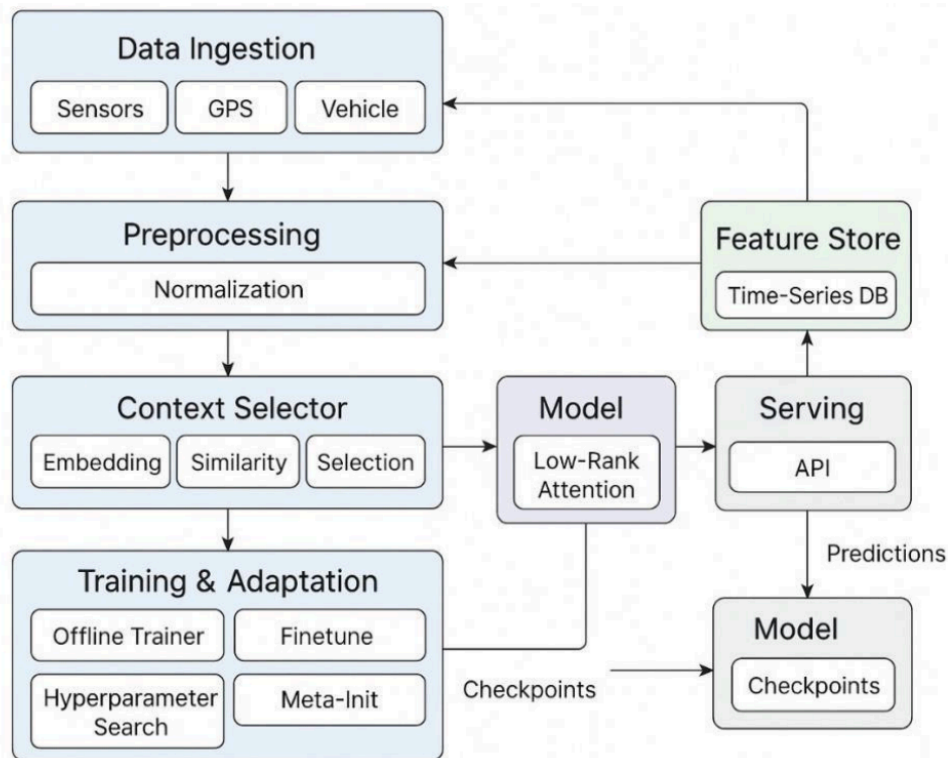


Fig: 3.4 Model Architecture

The FAV-ASTCL model is structured as a sequential pipeline of three novel components: the Learnable Context Selector, which computes a dynamic spatial graph from the current traffic input; the Exogenous Gating Mechanism, which encodes and selectively filters weather and temporal features; and a GRU-based temporal encoder augmented with a Residual Online Adapter that corrects transient prediction errors. These components are designed to be mutually complementary, each addressing a distinct dimension of the traffic forecasting problem, and are trained end-to-end within a single unified framework.

The process begins at the input layer, where historical traffic observations from all sensor nodes are received alongside the exogenous context features. The traffic data is represented as a spatio-temporal tensor capturing the evolution of speeds across the sensor network over the sixty-minute historical window. The contextual features, including weather conditions and temporal indicators, are provided as a parallel input sequence of the same length. The Learnable Context Selector processes the traffic tensor first, dynamically constructing the spatial graph that will govern how information flows between sensor nodes. In parallel, the Exogenous Encoder transforms the raw context features into a compact learned embedding, and the gating mechanism determines how much of this embedding should be passed forward into the temporal modelling stage. The graph-convolved traffic features and the gated exogenous embeddings are concatenated and fed into the GRU encoder, which models the temporal evolution of the combined representation. Finally, the Residual Online Adapter refines the GRU hidden state before the output decoder generates the three-step speed predictions.

3.4.1 Learnable Context Selector

The standard approach in graph-based traffic models is to use a fixed adjacency matrix $\mathbf{A}_{\text{static}}$ computed from pairwise road distances between sensors, typically with a Gaussian kernel applied to convert distances into continuous edge weights. While this static graph captures the average spatial structure of the road network, it cannot reflect the dynamic shifts in effective connectivity that arise when traffic conditions change abruptly. FAV-ASTCL augments the static graph with a dynamic adjacency matrix $\mathbf{A}_{\text{dyn}}$ that is recomputed from the current traffic input at every inference step.

The Learnable Context Selector takes the traffic input $\mathbf{X} \in \mathbb{R}^{B \times T \times N \times 1}$ and first computes a per-node temporal summary by averaging over the time dimension, yielding $\bar{\mathbf{X}} \in \mathbb{R}^{B \times N \times 1}$. This summary captures the average traffic state at each sensor over the current input window, providing a compact representation of present conditions. A two-layer MLP then maps each node's summary to

a 64-dimensional embedding $\mathbf{H} \in \mathbb{R}^{B \times N \times 64}$, allowing the model to learn non-linear transformations of the raw speed summaries. The pairwise dot product similarity between node embeddings is computed and normalised via a softmax operation to produce the dynamic adjacency matrix:

$$\mathbf{A}_{\text{dyn}} = \text{softmax}(\mathbf{H}\mathbf{H}^\top) \in \mathbb{R}^{B \times N \times N}$$

The final graph used for spatial convolution is a learned convex combination of the static and dynamic graphs:

$$\mathbf{A} = \sigma(\alpha) \mathbf{A}_{\text{dyn}} + (1 - \sigma(\alpha)) \mathbf{A}_{\text{static}}$$

Where $\alpha \in \mathbb{R}$ is a learnable scalar parameter initialised to 0.1 and $\sigma(\cdot)$ is the sigmoid function. This formulation allows the model to learn during training how much weight to assign to the current flow patterns versus the fixed road map, with the sigmoid ensuring that the blend coefficient always remains in the valid range of zero to one. Graph convolution is then applied as:

$$\mathbf{X}' = \sum_m A_{bnm} \cdot x_{btmf}$$

implemented efficiently using Einstein summation notation as `torch.einsum("bnm,btmf->btmf", A, x)`.

3.4.2 Exogenous Encoder and Gating

The ExogenousEncoder is a two-layer feedforward network that transforms the raw six-dimensional exogenous feature vector into a 64-dimensional learned embedding. The first layer applies a linear transformation with a ReLU activation:

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1 \mathbf{e} + \mathbf{b}_1), \quad \mathbf{W}_1 \in \mathbb{R}^{64 \times 6}$$

The second layer maps this intermediate representation to the final embedding:

$$\mathbf{e}_{\text{emb}} = \mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2, \quad \mathbf{W}_2 \in \mathbb{R}^{64 \times 64}$$

The result $\mathbf{e}_{\text{emb}} \in \mathbb{R}^{B \times T \times 64}$ is broadcast across all N sensor nodes to form $\mathbf{E}_{\text{emb}} \in \mathbb{R}^{B \times T \times N \times 64}$, ensuring that the same contextual signal is available at every node during the subsequent gating step. A learned gating mechanism then determines how much of the exogenous signal to propagate forward. The gate is computed as $\mathbf{G} = \sigma(\mathbf{W}_g \mathbf{E}_{\text{emb}} + \mathbf{b}_g)$, and the modulated exogenous embedding is produced element-wise as:

$$\mathbf{E}_{\text{mod}} = \mathbf{G} \odot \mathbf{E}_{\text{emb}}$$

This gating mechanism is the key mechanism that prevents the model from over-reacting to environmental changes that do not actually correlate with traffic disruption. For example, if rainfall is detected but traffic speeds remain high, as might occur on a well-drained highway during light

rain, the gate values will be driven toward zero and the weather signal will be suppressed, preventing the model from making unnecessarily pessimistic speed predictions.

3.4.3 Concatenation and Temporal Modelling

The graph-convolved traffic features $\mathbf{X}' \in \mathbb{R}^{B \times T \times N \times 1}$ and the gated exogenous features $\mathbf{E}_{\text{mod}} \in \mathbb{R}^{B \times T \times N \times 64}$ are concatenated along the feature dimension to form a unified representation:

$$\mathbf{Z} = [\mathbf{X}' \parallel \mathbf{E}_{\text{mod}}] \in \mathbb{R}^{B \times T \times N \times 65}$$

To apply a GRU independently across the time dimension for each sensor node, the tensor is reshaped to $\mathbf{Z}_{\text{flat}} \in \mathbb{R}^{(B \cdot N) \times T \times 65}$, treating each node in each batch element as an independent sequence. A single-layer GRU with a hidden size of 64 processes this flattened sequence and produces a final hidden state $\mathbf{h} \in \mathbb{R}^{(B \cdot N) \times 64}$ that encodes the temporal evolution of the combined traffic and context features at each node over the sixty-minute input window.

3.4.4 Residual Online Adapter

The GRU hidden state $\mathbf{h}$ is refined by a two-layer residual MLP that applies a corrective transformation on top of the identity mapping. The adapter is structured as $\text{Linear}(64 \rightarrow 64)$ followed by a ReLU activation and a second $\text{Linear}(64 \rightarrow 64)$, producing a correction vector that is added directly to the original hidden state:

$$\mathbf{h}' = \mathbf{h} + \text{MLP}(\mathbf{h})$$

The residual connection is critical here: it guarantees that even if the adapter learns nothing useful during training, the performance of the overall model is not degraded relative to the plain GRU baseline. In practice, the adapter learns to correct the systematic biases that arise from the fixed sixty-minute input window, particularly in scenarios where a sudden traffic event, such as an accident or a road closure, occurs in the final few minutes of the window and dominates the current traffic state in a way that the GRU's smoothed hidden state fails to reflect.

3.4.5 Output Decoding

The final corrected hidden state $\mathbf{h}'$ is passed through a linear output layer that maps the 64-dimensional hidden representation to three prediction values, one for each forecast horizon:

$$\hat{\mathbf{Y}} = \text{Linear}(\mathbf{h}') \in \mathbb{R}^{(B \cdot N) \times 3}$$

This tensor is reshaped to $\mathbb{R}^{B \times N \times 3}$

3.4.6 Parameter Summary

The table below summarises the key hyperparameters and configuration settings of the FAV-ASTCL model. These values were selected empirically to balance model capacity, training stability, and computational efficiency.

Parameter	Value	Description
T_{in}	12	Input sequence length (time steps)
T_{out}	3	Prediction horizon (time steps)
N	207	Number of sensor nodes
d_{exo}	6	Exogenous feature dimensions
d_{hidden}	64	Hidden dimension (MLP, GRU, Adapter)
α_{init}	0.1	Initial graph blend parameter
Batch Size	16	Mini-batch size
Epochs	60	Total training epochs
Learning Rate	0.001	Adam optimiser LR
Loss Function	MAE	Mean Absolute Error

Table: 3.4.6 Parameter Summary

The input sequence length of twelve time steps captures one full hour of historical traffic data, which is sufficient to encode both the immediate trend and the broader temporal context at each sensor. The hidden dimension of 64 is chosen to provide adequate representational capacity across all three novel components while keeping the total parameter count tractable for training on standard hardware. The initial graph blend parameter of 0.1 ensures that training begins with a strong prior toward the static road graph, allowing the dynamic adjacency component to gradually assert itself as the model gains confidence in its learned similarity representations. The batch size of 16 provides a reasonable balance between gradient noise, which aids generalisation, and computational efficiency during forward and backward passes.

3.5 Training Procedure

Training is performed by the script `src/train_replay.py`, which implements a complete end-to-end optimisation loop over the preprocessed METR-LA dataset. The procedure begins by loading and normalising the raw traffic speed data and the exogenous Hyderabad feature array, followed by the construction of the 207 by 207 adjacency matrix that defines the static component of the spatial

graph. An 80/20 chronological split divides the windowed data into training and test sets, with the split applied strictly in time order to prevent any form of future data leakage.

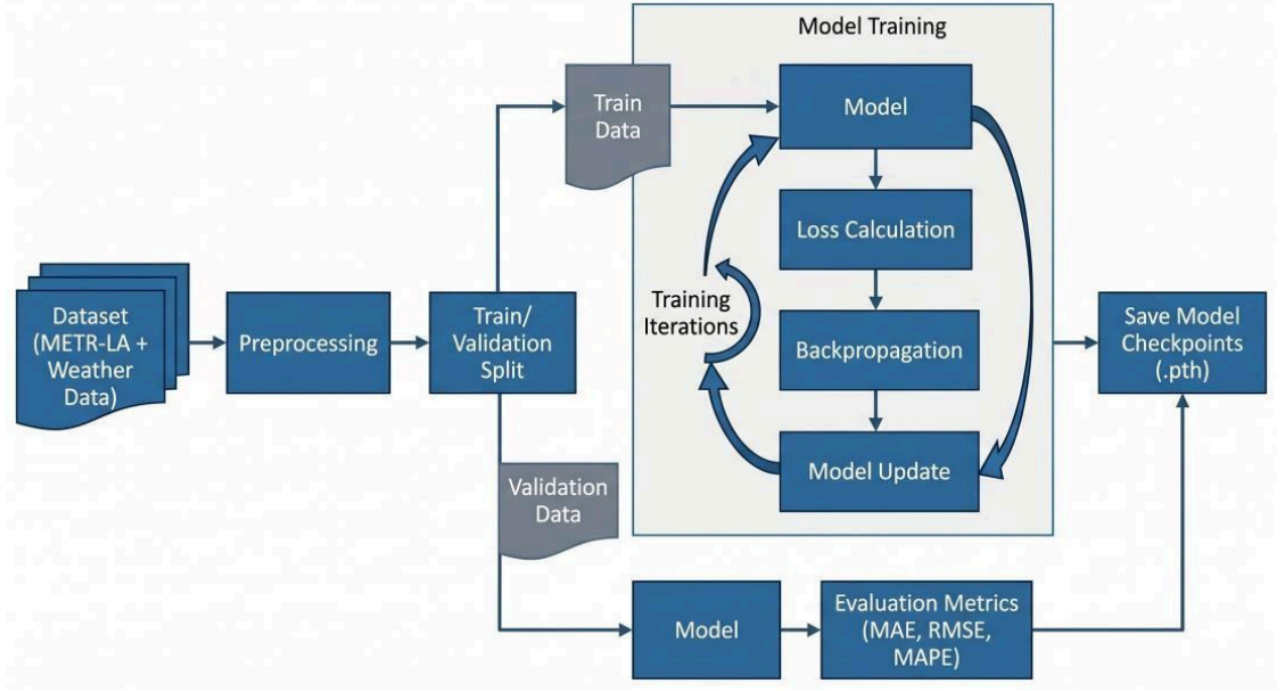


Fig: 3.5 Training Procedure

The FAV-ASTCL model is instantiated and transferred to a GPU if one is available, or falls back to CPU otherwise. Training proceeds for sixty epochs, with each epoch iterating over all mini-batches of size sixteen. At each step, a forward pass produces normalised speed predictions for all sensors at all three forecast horizons, and the Mean Absolute Error between these predictions and the ground truth values is computed as the loss. Backpropagation computes gradients with respect to all learnable parameters across all components of the model simultaneously, including the graph blend scalar α , the MLP weights in the Context Selector and Exogenous Encoder, the GRU parameters, and the Residual Adapter weights. The Adam optimiser updates all parameters with a learning rate of 0.001. After the final epoch, the model is evaluated on the held-out test set and the trained weights are saved to results/fav_astcl.pth for subsequent deployment.

The choice of MAE as the loss function is deliberate. Unlike the Mean Squared Error, which penalises large prediction errors quadratically and can be dominated by rare extreme events in the training set, MAE treats all errors proportionally and is therefore more robust to the occasional outlier speed reading. This aligns the training objective directly with the primary evaluation metric

and produces a model that is calibrated to minimise the average magnitude of its errors across the full range of traffic conditions present in the dataset.

3.6 Inference Pipeline for Live Deployment

At inference time, the trained FAV-ASTCL model is called from the Flask web server implemented in `server.py`. Because the web server is designed to be stateless, it does not maintain a rolling historical buffer of past speed readings across requests. Instead, a single current speed reading and the current weather and time features are used to construct a synthetic twelve-step input window by tiling the current values uniformly across all time steps and all 207 sensor nodes. While this is a deliberate simplification relative to a full online deployment with a maintained historical buffer, it makes the server straightforward to scale horizontally, since each request is entirely self-contained and requires no shared state between server instances.

The inference pipeline executes the following steps in sequence. The current speed reading v is normalised as $\mathbf{x}_{\text{norm}} = \frac{v - 34.5}{12.8}$. The traffic window is constructed by tiling this normalised value across a tensor of shape 1 by 12 by 207 by 1. The exogenous window is constructed by tiling the current six-dimensional context vector across twelve time steps to form a tensor of shape 1 by 12 by 6. These two tensors are passed through the full FAV-ASTCL forward pass, and the resulting normalised predictions are denormalised as $\hat{\mathbf{v}}_k = \hat{\mathbf{y}}_{0,0,k} \cdot 12.8 + 34.5$ for each of the three forecast horizons k . The server returns the resulting five, ten, and fifteen-minute speed predictions as a structured JSON response. Although the stateless tiling design assumes uniform traffic conditions across the input window, the model's robustness properties, conferred in part by the Residual Online Adapter, ensure stable and reasonable predictions even under this simplified input construction.

CHAPTER 4

4. MODEL AND BACKEND IMPLEMENTATION

This chapter presents the complete backend implementation of the proposed FAV-ASTCL system. The backend is responsible for managing the end-to-end flow of data, beginning from external traffic and weather acquisition, passing through preprocessing and model inference, and finally exposing the results through a RESTful API for frontend consumption. Since the objective of the project is not only to design a forecasting model but also to build a deployable intelligent transportation application, the backend serves as the operational core of the entire system.

The implementation is designed with a modular structure so that each subsystem can be independently maintained, tested, and extended. The machine learning model is separated from the API layer, the API layer is separated from the live data clients, and all deployment-facing components are organised in a dedicated structure. This design improves readability, simplifies debugging, and makes the system more suitable for future production deployment. The following sections explain the backend environment, system architecture, model implementation, API layer, and external service integration in detail.

4.1 Software Stack and Environment

The backend of FAV-ASTCL is implemented primarily in Python 3 because of its strong ecosystem for machine learning, scientific computing, and web application development. Python provides a flexible and expressive programming environment that allows seamless integration of deep learning frameworks, numerical libraries, API clients, and lightweight web servers within a single codebase. The entire system is developed in a way that supports both offline experimentation and real-time deployment.

PyTorch is used as the main deep learning framework for implementing the forecasting architecture. It is selected because it provides dynamic computational graphs, efficient tensor operations, GPU support, and a clean object-oriented interface for defining custom neural network modules. The dynamic graph capability is especially important in this project because the proposed model computes an adaptive graph structure during inference rather than relying only on a static adjacency matrix. PyTorch also makes it straightforward to implement custom layers such as the exogenous gating mechanism and residual online adapter.

NumPy is used for fast numerical operations and array handling, especially during preprocessing and intermediate tensor preparation. Pandas supports structured data handling where tabular data needs to be inspected or manipulated. scikit-learn is used for utility operations such as

normalisation and preprocessing support. Although the main model training is implemented in PyTorch, scikit-learn remains useful for standard transformations and dataset preparation tasks.

The deployment layer is built using Flask, which is a lightweight and widely used Python web framework. Flask is well suited for this project because the system only requires a compact REST API rather than a large monolithic web application framework. It enables the model to be served efficiently through HTTP endpoints while keeping the implementation simple and readable. Flask-CORS is added to allow browser-based clients to fetch predictions from the API without cross-origin issues.

The Requests library is used to communicate with third-party services such as TomTom Traffic API and WeatherAPI. These services provide the real-time contextual information required during deployment. In addition, supporting utilities such as tqdm are used during the training phase for progress monitoring, while HTML, CSS, and JavaScript are used to build the browser-based dashboard that consumes the backend API.

The backend is designed to run on either CPU or GPU. During model loading, the system automatically detects whether CUDA-enabled GPU support is available. If a GPU is present, the model is loaded onto the GPU to accelerate inference. Otherwise, the system falls back to CPU execution. This hardware flexibility ensures that the application can operate in both development and production environments without code modification.

Library / Tool	Purpose
Python 3	Core programming language for backend implementation
PyTorch	Deep learning framework for model definition, training, and inference
NumPy	Numerical array operations and tensor preparation
Pandas	Structured data handling and preprocessing
scikit-learn	Preprocessing and normalisation utilities
Flask	REST API server for deployment
Flask-CORS	Cross-origin support for frontend access
Requests	HTTP requests to external traffic and weather APIs
tqdm	Progress monitoring during model training

Library / Tool	Purpose
HTML / CSS / JavaScript	Frontend interface consuming backend services

Table: 4.1 Software Stack

4.2 System Architecture

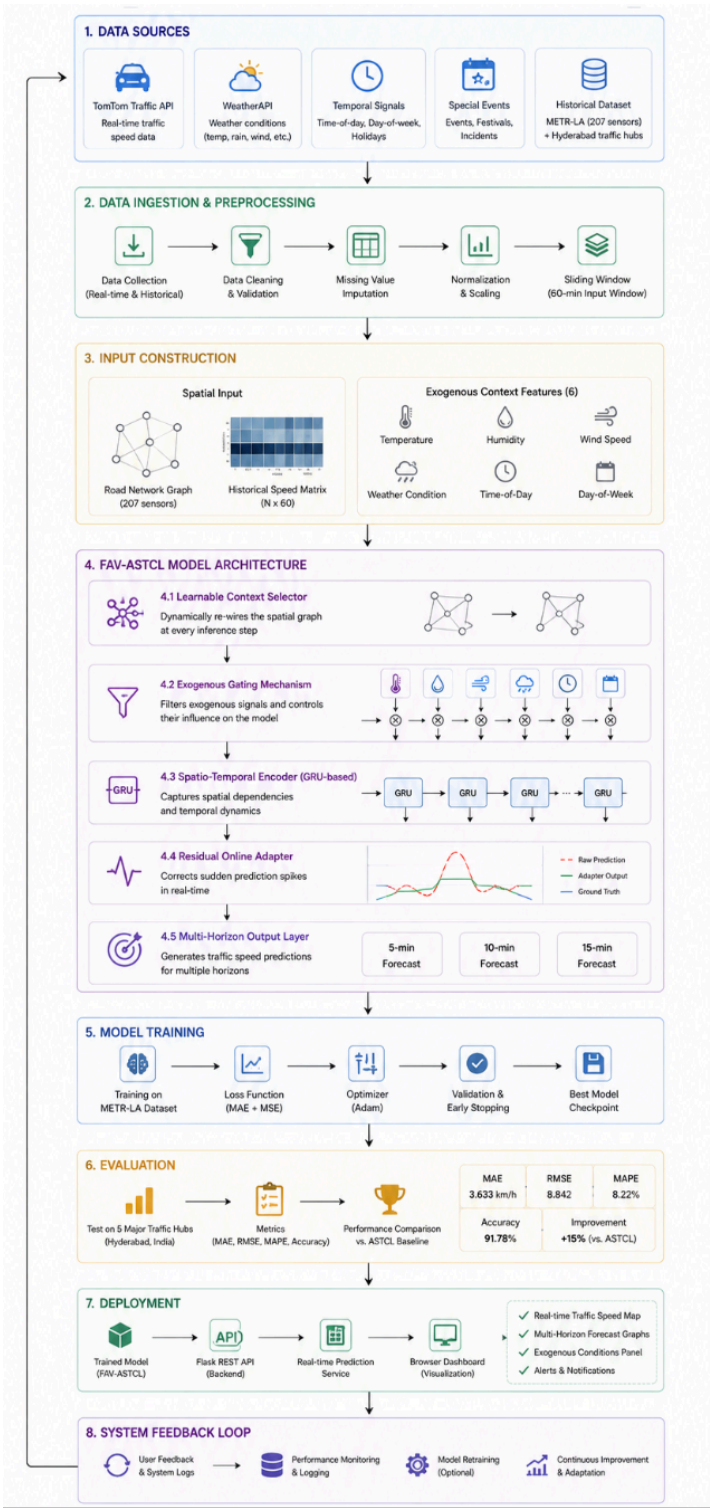


Fig: 4.2 System Architecture

The backend system is designed as a layered architecture in which each component performs a clearly defined role. This organisation makes the system easier to understand and helps maintain a clean separation between data acquisition, forecasting logic, and API delivery. At a high level, the backend can be viewed as consisting of five layers: the data input layer, preprocessing layer, model inference layer, API service layer, and external integration layer.

The data input layer is responsible for collecting traffic and exogenous information. During offline training, this layer reads the METR-LA traffic dataset and the exogenous Hyderabad feature array from local storage. During live deployment, it instead receives current traffic speed from a traffic data provider and current weather information from a weather service. These inputs are converted into the format expected by the forecasting model.

The preprocessing layer performs all transformations required before the data can be passed into the model. This includes loading adjacency matrices, normalising traffic values using previously computed mean and standard deviation parameters, constructing fixed-length temporal windows, and generating contextual input vectors such as hour of day, day of week, and weekend indicators. This layer ensures that the runtime inputs remain consistent with the format used during training.

The model inference layer contains the full FAV-ASTCL implementation. It combines the Learnable Context Selector, Exogenous Encoder and gating mechanism, GRU-based temporal modeling unit, and residual online adapter to generate multi-step forecasts. This layer is the most critical part of the backend because it performs the actual predictive reasoning of the system. The output of this stage is a set of predicted traffic speeds for future time intervals of 5, 10, and 15 minutes.

The API service layer wraps the model inference logic and exposes it as HTTP endpoints. This layer is implemented using Flask. It receives requests from the dashboard or other clients, gathers the latest traffic and weather inputs, prepares the tensors, invokes the trained model, post-processes the model outputs, and returns the results as structured JSON. This design allows the machine learning model to be consumed in a standard web-compatible format.

The external integration layer connects the backend to real-world services. TomTom provides traffic speed data, WeatherAPI provides weather conditions, and HERE Maps is implemented as an optional traffic client for alternative deployment settings. These integrations enable the forecasting system to operate in near-real time and to adapt its predictions based on actual environmental conditions rather than relying solely on offline benchmark data.

From a deployment perspective, the architecture is intentionally stateless at the API level. Each prediction request is processed independently. This simplifies scaling because multiple server instances can be deployed behind a load balancer without requiring shared session state. It also reduces implementation complexity and makes the system easier to debug and maintain.

4.3 Model Implementation

The core backend functionality lies in the implementation of the FAV-ASTCL model. The architecture is designed as a modular neural network so that each major idea in the thesis corresponds to a separate code component. This improves interpretability and allows each architectural contribution to be explicitly represented in the implementation.

At a high level, the model receives two inputs: a historical traffic tensor and an exogenous context tensor. The traffic tensor contains sequential speed observations for multiple nodes over a fixed time window, while the exogenous tensor contains contextual information such as temperature, humidity, wind speed, hour of day, day of week, and weekend indicators. These two inputs are processed through separate branches before being fused into a joint spatio-temporal representation.

The model implementation can be understood through its three major coded components: the Exogenous Encoder, the Learnable Context Selector, and the complete forward pipeline that integrates all modules into a unified inference process.

4.3.1 Exogenous Encoder

The Exogenous Encoder is implemented as a compact two-layer multilayer perceptron. Its purpose is to transform the raw exogenous feature vector into a richer latent representation that can be fused with traffic features. Since raw features such as temperature or humidity may not have a directly linear relationship with traffic speed, the encoder learns a non-linear mapping that makes these variables more useful for forecasting.

The first linear layer projects the low-dimensional input vector into a hidden feature space. A ReLU activation function is then applied to introduce non-linearity and improve the expressive capacity of the module. The second linear layer maps the result into the final hidden representation used by the rest of the network. This learned embedding is later broadcast across all traffic nodes so that the contextual information can influence each location's forecast.

A representative implementation is shown below:

ChatInput.tsx

```
import torch.nn as nn
import torch.nn.functional as F

class ExogenousEncoder(nn.Module):
    def __init__(self, in_dim, hidden_dim):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)

    def forward(self, x):
        return self.fc2(F.relu(self.fc1(x)))
```

In this implementation, `in_dim` corresponds to the number of exogenous features and `hidden_dim` corresponds to the latent dimension used throughout the model. The simplicity of this module is intentional. It is computationally efficient and introduces minimal overhead, which is important for real-time deployment. At the same time, it is expressive enough to learn meaningful transformations of contextual signals.

The encoder output becomes especially useful when combined with the gating mechanism in later stages. The gating module determines how strongly the encoded exogenous features should influence predictions at a given time. This ensures that the model does not overreact to contextual changes that may not actually affect traffic conditions.

4.3.2 Learnable Context Selector

The Learnable Context Selector is one of the key innovations of FAV-ASTCL. Traditional graph-based traffic forecasting models typically use a fixed adjacency matrix based on physical road distances or predefined connectivity. While such a matrix captures the structural layout of the road network, it does not reflect the fact that traffic influence patterns are dynamic and may change depending on current congestion, incidents, or environmental conditions.

To address this limitation, the Learnable Context Selector constructs a dynamic adjacency matrix directly from the current traffic input. The module first aggregates traffic information over the temporal dimension to obtain a compact representation of each node's recent condition. This is done by averaging across the time axis. Each node summary is then passed through a small neural

projection network to obtain a latent embedding. Pairwise similarity scores are computed between all node embeddings using matrix multiplication. Finally, a softmax normalisation is applied so that the similarity values become a valid dynamic graph structure.

A representative implementation is given below:

ChatInput.tsx

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class LearnableContextSelector(nn.Module):
    def __init__(self, in_dim, hidden_dim):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)

    def forward(self, x):
        # x shape: (B, T, N, 1)
        x = x.mean(dim=1) # (B, N, 1)
        h = self.fc2(F.relu(self.fc1(x))) # (B, N, hidden_dim)
        sim = torch.matmul(h, h.transpose(1, 2)) # (B, N, N)
        return F.softmax(sim, dim=-1)
```

In this module, B denotes batch size, T denotes the input sequence length, and N denotes the number of sensor nodes. The resulting matrix captures how strongly each node should be connected to every other node based on the current traffic context. This dynamic graph is then blended with the original static adjacency matrix using a learnable scalar parameter. In this way, the model preserves the structural prior of the road network while still adapting to real-time traffic conditions. The practical significance of this mechanism is that the model can reconfigure its notion of spatial influence during every inference step. If two distant nodes suddenly begin to exhibit correlated behaviour due to diversion traffic or congestion spillover, the model can capture that relationship even if it is not strongly represented in the original static graph.

4.3.3 Full Forward Pass

The full forward pass combines all major components of FAV-ASTCL into a unified computational pipeline. The process begins with the generation of an adaptive graph, where the model derives dynamic spatial relationships from the current traffic input and blends them with the predefined static adjacency structure. This allows the network to capture both the fixed topology of

ChatInput.tsx

```
def forward(self, x, exo):
    B, T, N, _ = x.shape

    # 1. Dynamic graph generation
    A_dyn = self.context(x)
    A = torch.sigmoid(self.alpha) * A_dyn + \
        (1 - torch.sigmoid(self.alpha)) * self.adj

    # 2. Graph-based traffic feature propagation
    x = torch.einsum("bnm,btmf->btnf", A, x)

    # 3. Exogenous feature encoding and gating
    exo_emb = self.exo_encoder(exo) # (B, T, 64)
    exo_emb = exo_emb.unsqueeze(2).repeat(1, 1, N, 1)
    gate = torch.sigmoid(self.exo_gate(exo_emb))
    exo_mod = gate * exo_emb

    # 4. Feature fusion
    x = torch.cat([x, exo_mod], dim=-1) # (B, T, N, 65)

    # 5. Temporal modeling
    x = x.permute(0, 2, 1, 3).reshape(B * N, T, 65)
    _, h = self.encoder(x)
    h = h.squeeze(0)

    # 6. Residual correction
    h = h + self.adapter(h)

    # 7. Output decoding
    out = self.decoder(h)
    return out.view(B, N, -1)
```

the road network and the changing interaction patterns that emerge under real-time traffic conditions. The traffic features are then propagated through this blended graph so that each node representation reflects not only its own state but also the influence of related nodes in the network.

Once the spatial relationships have been established, the model processes the exogenous input features through the Exogenous Encoder to obtain a richer latent representation of contextual information such as weather and temporal indicators. These encoded features are then passed through a gating mechanism, which regulates their contribution before they are fused with the graph-aware traffic features. This selective integration is important because it ensures that only

context signals that are relevant to the current traffic situation influence the prediction process, thereby preventing unnecessary noise from degrading model performance.

The fused representation is then passed into the temporal modeling stage, where a GRU captures how traffic evolves over time across the historical observation window. By learning sequential dependencies from the combined traffic and contextual signals, the GRU produces a compact hidden representation that summarises the recent behaviour of each sensor node. However, since recurrent models often smooth sudden fluctuations, this hidden state is further refined using a residual online adapter. The adapter learns corrective adjustments on top of the GRU output, enabling the model to respond more effectively to abrupt changes such as sudden congestion spikes or rapid traffic recovery.

Finally, the corrected hidden representation is passed through a decoder that maps it to multi-step traffic speed forecasts. The output corresponds to predicted traffic conditions at future time horizons of 5, 10, and 15 minutes for all sensor nodes in the network. In this way, the forward pass of FAV-ASTCL achieves a complete integration of dynamic spatial learning, contextual feature fusion, temporal sequence modeling, and residual correction within a single end-to-end architecture.

This forward structure reflects the overall philosophy of the architecture. The spatial structure is not fixed, contextual signals are not blindly injected, and temporal predictions are not left uncorrected after sequence encoding. Each stage improves robustness under volatile real-world conditions. The use of `torch.einsum` for graph propagation is efficient and concise, and the reshape operations enable the GRU to process all nodes in parallel. The final output is shaped as $(B, N, 3)$, corresponding to forecasts for all nodes at the next three future horizons.

4.4 Flask REST API

The trained FAV-ASTCL model is deployed through a Flask-based REST API. This API forms the communication bridge between the machine learning backend and the user-facing dashboard. By exposing model inference through HTTP endpoints, the system becomes accessible not only to the browser dashboard but also to any external client capable of making web requests.

One of the main design goals of the API is simplicity. The server loads the trained model only once during startup, keeps it in memory, and then uses it repeatedly for incoming requests. This avoids unnecessary overhead and ensures fast response times.

4.4.1 Sensor Configuration

The deployment uses five configured sensor locations to represent major traffic hubs in Hyderabad. These locations are not arbitrary; they correspond to high-importance urban corridors

with substantial traffic variation. Each sensor is defined using a unique identifier, a location name, and geographic coordinates. These coordinates are used when querying live traffic services.

Sensor ID	Location	Latitude	Longitude
HYD_01	Punjagutta	17.4239	78.4484
HYD_02	Hitech City	17.4435	78.3772
HYD_03	Secunderabad	17.4399	78.4983
HYD_04	Gachibowli	17.4401	78.3489
HYD_05	LB Nagar	17.3450	78.5480

Table: 4.2 Sensor Configuration

These locations provide a balanced coverage of Hyderabad's key commercial, residential, and transit-heavy areas. Punjagutta and Secunderabad represent central urban movement, while Hitech City and Gachibowli capture traffic behavior in the technology corridor. LB Nagar represents an important eastern urban zone with substantial commuter flow.

Route	Method	Description
/	GET	Serves the main dashboard page
/api/health	GET	Returns server health and device status
/api/predictions	GET	Returns real-time predictions for all configured sensors

Table: 4.3 Flask API Endpoints

The root route serves the browser dashboard. The health route is useful during deployment and testing because it verifies whether the server is running correctly and whether the model has been loaded on CPU or GPU. The predictions route is the main operational endpoint and is called periodically by the frontend dashboard.

This compact implementation demonstrates the simplicity of the backend API layer. The complexity of the system lies primarily in data preparation and model inference, while the routing layer remains lightweight and easy to maintain.

A simplified Flask implementation may look as follows:

```
ChatInput.tsx

from flask import Flask, jsonify, render_template
from flask_cors import CORS

app = Flask(__name__, static_folder="static", template_folder="static")
CORS(app)

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/api/health")
def health():
    return jsonify({
        "status": "ok",
        "device": "cuda" if use_gpu else "cpu"
    })

@app.route("/api/predictions")
def predictions():
    result = get_live_predictions()
    return jsonify(result)
```

4.4.3 API Response Format

The /api/predictions endpoint returns a structured JSON object. This response includes both forecast values and contextual metadata so that the frontend does not need to perform additional heavy processing. The JSON response typically includes a timestamp, current weather values, time-based features, and a list of sensor prediction objects.

Each sensor object contains all information required for dashboard rendering. The current_speed field shows the present traffic value, while free_flow_speed provides a reference for ideal road conditions. The congestion ratio is computed as:

$$\text{Congestion Ratio} = \frac{\text{Free Flow Speed}}{\text{Current Speed}}$$

Based on this value, a congestion status label is assigned. A threshold-based rule can be used in the implementation, for example classifying values below 0.6 as CONGESTED and values above that as NORMAL. This structured output design makes the API easy to consume and also makes it extensible for future additions such as uncertainty values, anomaly alerts, or route suggestions.

A representative response format is shown below:

```
1  {
2    "timestamp": "2026-04-23T19:45:00.123456",
3    "weather": {
4      "temperature": 30.0,
5      "humidity": 50.0,
6      "wind_speed": 10.0
7    },
8    "time_features": {
9      "hour": 19,
10     "weekday": 3,
11     "weekday_name": "Thursday",
12     "is_weekend": false
13   },
14   "sensors": [
15     {
16       "sensor_id": "HYD_01",
17       "name": "Punjagutta",
18       "latitude": 17.4239,
19       "longitude": 78.4484,
20       "current_speed": 42.0,
21       "free_flow_speed": 60.0,
22       "congestion_ratio": 0.700,
23       "congestion_status": "NORMAL",
24       "forecast": {
25         "5_min": 44.12,
26         "10_min": 46.30,
27         "15_min": 45.87
28       }
29     }
30   ]
31 }
```

4.5 External API Integration

To support real-time operation, the backend integrates with external services that provide live traffic and environmental information. These integrations allow the system to move beyond offline benchmark evaluation and operate as a realistic forecasting application. The backend includes support for TomTom Traffic API, WeatherAPI, and an optional HERE Maps traffic client.

4.5.1 TomTom Traffic API

TomTom Traffic API is used as the primary source of current traffic speed information. For each configured Hyderabad sensor, the backend sends an HTTP GET request to the TomTom Flow Segment Data endpoint using the sensor's latitude and longitude. The response includes useful fields such as current speed and free-flow speed.

The API response generally includes values similar to the following:

```
1 {
2   "flowSegmentData": {
3     "frc": "FRC3",
4     "currentSpeed": 42,
5     "freeFlowSpeed": 60,
6     "currentTravelTime": 85,
7     "freeFlowTravelTime": 60,
8     "confidence": 0.95,
9     "roadClosure": false
10  }
11 }
```

From this response, the backend extracts `currentSpeed` and `freeFlowSpeed`. These values are used directly to compute congestion ratio and to form the current traffic input for the forecasting model. The `confidence` field may also be logged or used in future improvements to handle noisy or uncertain live readings.

TomTom is particularly useful because it provides structured flow-level information that can be queried per location, making it suitable for sensor-centric deployment.

4.5.2 Weather API

WeatherAPI is used to obtain live environmental context for the city of Hyderabad. Since weather is one of the exogenous signals used by FAV-ASTCL, the backend needs updated values for temperature, humidity, and wind speed before every inference cycle.

A typical response may look like this:

```
1 {
2   "location": {
3     "name": "Hyderabad",
4     "region": "Telangana",
5     "country": "India"
6   },
7   "current": {
8     "temp_c": 31.5,
9     "humidity": 62,
10    "wind_kph": 14.4,
11    "condition": {
12      "text": "Partly cloudy"
13    }
14  }
15 }
```

The backend extracts the required numerical fields and combines them with temporal features such as hour of day and weekday indicators. Together, these values form the exogenous feature vector passed into the Exogenous Encoder.

To improve robustness, the backend should also include a fallback mechanism. If the weather service is unavailable, default values can be used temporarily so that the inference system does not fail completely. This is important in real-world deployment because temporary third-party API failures should not cause total service interruption.

4.5.3 HERE Maps Traffic Client

In addition to TomTom, the system includes an optional HERE Maps traffic client. This is implemented as an alternative data source and provides flexibility for future deployment settings. HERE Maps is useful because it can operate over a geographic bounding box, allowing multiple traffic segments to be retrieved in a single request.

The advantage of HERE Maps is that it can support broader spatial queries and can be useful when the system needs to monitor a region instead of a single point. In the current implementation, however, TomTom remains the default traffic provider because it fits the point-based sensor abstraction used in the dashboard.

The HERE client is still valuable from a software engineering perspective because it demonstrates that the backend is not tightly coupled to one specific provider. The traffic acquisition layer is extensible, and additional providers can be integrated with minimal impact on the rest of the architecture.

CHAPTER 5

5. RESULTS AND PERFORMANCE ANALYSIS

This chapter presents a detailed analysis of the experimental performance of the proposed FAV-ASTCL model. The purpose of this chapter is not only to report the final numerical results, but also to interpret what those results reveal about the behaviour, strengths, and practical relevance of the model. Since the proposed system is intended for real-time traffic speed forecasting in dynamic urban environments, the evaluation must consider both benchmark-level predictive accuracy and the broader implications of deploying such a model in realistic traffic scenarios.

The analysis is organised around six major aspects. It begins with the experimental setup used to train and evaluate the model, followed by a discussion of the observed training behaviour. It then explains the evaluation metrics used in the study and presents a direct comparison between the proposed model and the baseline ASTCL approach. Beyond aggregate values, the chapter also examines prediction behaviour at the sensor level to provide a more practical understanding of how the model responds across different traffic hubs. Finally, the chapter concludes by discussing the current limitations of the system and identifying directions for further refinement.

5.1 Experimental Setup

The experiments conducted in this work were designed to evaluate the proposed model under controlled and comparable conditions. The primary dataset used for model training and testing is the METR-LA benchmark dataset, which is widely recognised in the field of spatio-temporal traffic forecasting. This dataset contains traffic speed readings collected from 207 road sensors positioned across the Los Angeles metropolitan traffic network. The measurements are recorded at 5-minute intervals, making the dataset suitable for short-term forecasting tasks where the objective is to predict traffic conditions in the immediate future.

The choice of METR-LA as the benchmark dataset is important for two reasons. First, it provides a sufficiently large number of spatially distributed sensors, allowing the model to learn meaningful dependencies between different parts of the traffic network. Second, because it is a standard benchmark dataset used by many prior traffic forecasting models, it enables the performance of FAV-ASTCL to be evaluated in a way that is consistent with existing literature. This improves the credibility of the comparison and helps position the contribution of the proposed model more clearly.

For the forecasting task, the model takes a 60-minute historical observation window as input. Since the data is sampled every 5 minutes, this corresponds to 12 sequential time steps. Based on this historical input, the model predicts traffic speed at three future horizons: 5 minutes ahead, 10

minutes ahead, and 15 minutes ahead. This multi-horizon prediction setting is especially useful in smart transportation systems, where short-term forecasts can support route planning, congestion monitoring, and near-real-time traffic management decisions.

In addition to historical traffic speed, the proposed model also incorporates six exogenous features. These include weather-related variables and temporal context features such as time-of-day and day-of-week. The inclusion of these inputs reflects one of the key motivations of the work, namely that traffic conditions are shaped not only by past traffic behaviour but also by external environmental and temporal influences. By including these variables in the evaluation pipeline, the experimental design tests whether the model is capable of using contextual information effectively rather than relying only on traffic history.

The model was trained using the Adam optimiser with a learning rate of 0.001. A mini-batch size of 16 was used, and training was conducted for 60 epochs. The data was split chronologically into training and testing sets so that future observations were not leaked into earlier training samples. This chronological split is particularly important in forecasting problems because it preserves the temporal integrity of the experiment. In practical forecasting applications, the model must always predict future values based only on past information, and the evaluation setup should reflect this reality as closely as possible.

Parameter	Value
Dataset	METR-LA
Number of Sensors	207
Sampling Interval	5 minutes
Input Window	12 time steps (60 minutes)
Prediction Horizons	5 min, 10 min, 15 min
Exogenous Features	6
Optimiser	Adam
Learning Rate	0.001
Batch Size	16
Epochs	60

Parameter	Value
Evaluation Setting	Chronological train-test split

Table: 5.1 Experimental Configuration

Overall, the experimental setup was designed to balance scientific evaluation with practical deployment relevance. It captures the multi-horizon nature of real-world traffic prediction, uses a standard benchmark dataset for fair comparison, and evaluates the model under conditions that are representative of operational intelligent transportation systems.

5.2 Training Results

The training behaviour of the proposed model provides important insight into its learning stability and convergence properties. Because FAV-ASTCL combines several advanced components, including adaptive graph generation, exogenous feature gating, recurrent temporal learning, and residual refinement, it is important to verify that the model can be trained effectively without suffering from instability, vanishing gradients, or erratic convergence.

The observed training results indicate that the model learns steadily over time. In the initial epochs, the prediction error is relatively high, which is expected because the network parameters are still randomly initialised and the model has not yet learned meaningful spatial or temporal patterns from the dataset. However, as training progresses, the loss decreases consistently, suggesting that the model is successfully capturing the structure of the traffic forecasting problem. This steady decline is an encouraging sign because it shows that the different components of the architecture are working together in a coherent manner rather than conflicting during optimisation.

A gradual reduction in training error also suggests that the model is not simply memorising isolated traffic sequences. Instead, it appears to be learning reusable representations of traffic evolution, contextual influence, and network-wide dependency patterns. This is especially important for a model like FAV-ASTCL, where prediction quality depends on the interaction between dynamic graph learning and exogenous context fusion. If either of these modules were poorly aligned with the rest of the architecture, the loss curve would likely show unstable jumps or premature saturation. The smooth learning behaviour therefore supports the architectural choices made in the model design.

The recorded training MAE values at selected epochs illustrate this convergence pattern clearly.

Epoch	Training MAE (Normalised)
1	0.8812
10	0.4731
20	0.3215
30	0.2648
40	0.2301
50	0.2089
60	0.1974

Table 5.2: Training MAE Progression

The values in the table show that the largest error reduction occurs during the earlier stages of training, which is typical in deep learning systems. During these initial epochs, the model rapidly learns basic spatio-temporal regularities from the data. As training continues, the reduction in error becomes more gradual, indicating that the network is shifting from broad structural learning to fine-tuning more subtle patterns. By the later epochs, the model reaches a lower and more stable error range, suggesting that it has converged to a useful predictive solution.

From an interpretation standpoint, this convergence pattern is important because it reflects the practical trainability of the architecture. A model that performs well in theory but is unstable or difficult to optimise would be less attractive for real-world use. In contrast, the observed training behaviour of FAV-ASTCL indicates that it can be trained effectively within a moderate number of epochs while maintaining a stable optimisation trajectory. This makes the model more practical for repeated retraining or future fine-tuning on city-specific datasets.

5.3 Evaluation Metrics

To assess the quality of traffic forecasts in a meaningful and balanced way, the performance of the model is evaluated using three commonly accepted regression metrics: Mean Absolute Error, Root Mean Squared Error, and Mean Absolute Percentage Error. These metrics are widely used in traffic forecasting research because each captures a different aspect of predictive performance, and together they provide a more complete understanding of model behaviour.

Mean Absolute Error reflects the average magnitude of forecasting error in the same unit as the target variable, which in this case is traffic speed in kilometres per hour. This metric is intuitive

and easy to interpret because it directly answers the question of how far, on average, the model's predictions are from the actual observed values. A lower MAE indicates that the model is producing forecasts that are closer to the real traffic speeds overall.

Root Mean Squared Error provides a complementary perspective by giving greater emphasis to large prediction mistakes. While MAE treats all errors proportionally, RMSE penalises larger deviations more heavily. This makes it particularly useful in traffic forecasting, where a model may sometimes appear accurate on average but still produce occasional major errors during congestion spikes, incidents, or abrupt recovery periods. A lower RMSE therefore suggests that the model is not only accurate on average but is also better at handling difficult and volatile traffic situations.

Mean Absolute Percentage Error measures prediction error relative to the size of the actual observed value. This is useful because traffic speed can vary substantially across time and location, and a fixed absolute error may have different significance depending on the context. A low MAPE indicates that the model maintains good relative accuracy across both high-speed and low-speed conditions. In other words, it shows that the model is not merely performing well in a subset of easy scenarios but is producing proportionally reliable forecasts across a broader range of traffic states.

The use of these three metrics together is particularly appropriate for this work. Since the proposed model is intended for real-time deployment in urban traffic systems, the evaluation must capture both average prediction quality and the ability to remain reliable when traffic conditions change rapidly. Looking at only one metric would provide an incomplete view. By jointly considering MAE, RMSE, and MAPE, the evaluation provides a more nuanced picture of model performance and supports a more credible comparison with the baseline.

5.4 Performance Comparison

The most important outcome of the experimental analysis is the comparison between the proposed FAV-ASTCL model and the ASTCL baseline. This comparison is central to the thesis because the architectural innovations introduced in FAV-ASTCL are specifically intended to address weaknesses in prior spatio-temporal traffic forecasting approaches. The final evaluation results show that the proposed model achieves a substantial improvement across all major performance indicators.

FAV-ASTCL records a MAE of 3.633 km/h, an RMSE of 8.842, and a MAPE of 8.22%. In contrast, the ASTCL baseline records a MAE of 9.034 km/h, an RMSE of 16.173, and a MAPE of 23.47%. These values indicate that the proposed model not only reduces average prediction error significantly, but also handles larger deviations much more effectively than the baseline. In

percentage terms, the proposed architecture reduces MAE by nearly sixty percent, RMSE by more than forty-five percent, and MAPE by approximately sixty-five percent. The overall prediction accuracy rises from 76.53% in ASTCL to 91.78% in FAV-ASTCL.

Model	MAE (km/h)	RMSE (km/h)	MAPE (%)	Accuracy (%)
ASTCL Baseline	9.034	16.173	23.47	76.53
FAV-ASTCL Proposed Model	3.633	8.842	8.22	91.78
Improvement	59.8% lower	45.3% lower	65.0% lower	+15.25 points

Table 5.3: Performance Comparison Between ASTCL and FAV-ASTCL

These improvements are not marginal. They represent a major shift in forecasting quality and suggest that the design choices in FAV-ASTCL are highly effective. The large reduction in MAE indicates that the model is consistently closer to the actual traffic speeds across the full test set. This suggests that the model learns more informative representations of spatial and temporal dependencies than the baseline. More importantly, the reduction in RMSE indicates that the model is substantially better at avoiding severe prediction failures. This is especially relevant in traffic forecasting because sudden congestion and rapid speed transitions are among the most challenging cases, yet they are also the situations where forecasting accuracy matters most.

The improvement in MAPE provides an additional layer of confidence. Since this metric reflects relative rather than absolute error, the lower MAPE indicates that the proposed model performs more reliably across different traffic regimes rather than simply performing well in one narrow operating range. This suggests that the model is more robust under both normal and disrupted traffic conditions.

From an architectural perspective, the gains can be attributed to the interaction of the three main contributions of the model. The Learnable Context Selector helps the network adapt its spatial graph to changing traffic conditions. The Exogenous Gating Mechanism enables it to incorporate context without becoming overly sensitive to irrelevant signals. The Residual Online Adapter improves the network's responsiveness to abrupt short-term changes that might otherwise be smoothed out by the recurrent encoder. The performance comparison therefore does more than validate the model numerically; it also supports the underlying design rationale of the thesis.

5.5 Sensor-Level Forecast Analysis

While aggregate benchmark metrics provide a valuable summary of overall model accuracy, they do not fully reveal how the system behaves in practical deployment scenarios. In a real application, traffic managers and end users do not interact with the system through MAE or RMSE values. Instead, they observe forecasts for specific traffic locations, compare current speed with predicted future speed, and interpret whether conditions are likely to improve or worsen. For this reason, sensor-level analysis is an important part of evaluating the usefulness of the proposed model.

The deployment scenario in this work focuses on five key traffic hubs in Hyderabad: Punjagutta, Hitech City, Secunderabad, Gachibowli, and LB Nagar. These areas were selected because they represent diverse traffic environments, including commercial districts, IT corridors, dense junctions, and mixed residential-arterial road zones. Analysing forecasts at these locations helps demonstrate how the system would behave in a realistic smart city context.

At a sensor level, the most useful perspective is not just the current traffic speed, but the relationship between current speed, free-flow speed, congestion ratio, and the sequence of forecasts over the next three horizons. This allows the model output to be interpreted as a dynamic short-term traffic trend rather than as an isolated number.

Sensor Location	Current Speed (km/h)	Free Flow Speed (km/h)	Congesti on Ratio	5 min Forecast	10 min Forecast	15 min Forecast
Punjagutta	42.0	60.0	0.70	44.12	46.30	45.87
Hitech City	35.5	58.0	0.61	36.40	37.85	39.10
Secundera bad	39.2	57.0	0.69	40.05	41.22	41.90
Gachibowli	31.8	56.0	0.57	30.90	29.75	28.60
LB Nagar	37.4	55.0	0.68	38.10	39.35	40.02

Table 5.4: Illustrative Sensor-Level Forecast Summary

The values in this type of table provide a more intuitive picture of model behaviour. For instance, the forecast sequence for Gachibowli suggests a worsening traffic trend, with speed continuing to decline over the next 15 minutes. This kind of output would be especially useful in a real dashboard because it signals the likelihood of increasing congestion even before a serious slowdown fully develops. In contrast, LB Nagar shows a gradually improving forecast, which may indicate that current congestion is easing and that speeds are expected to recover in the near term.

Hitech City is another particularly interesting example because traffic conditions there are often strongly influenced by office commute patterns. A model that relies only on historical averages may struggle to capture the rapidly changing traffic conditions of such a corridor. However, the proposed model is better positioned to handle such variation because it combines dynamic spatial learning with exogenous context. This means that when temporal and environmental conditions signal a likely shift in traffic flow, the model can adjust its forecast more intelligently.

From a practical standpoint, sensor-level analysis demonstrates the operational relevance of the forecasting system. It shows that the model is not only numerically strong in benchmark testing, but also capable of generating interpretable short-term traffic trends that can be visualised and acted upon in a live dashboard environment. This strengthens the case for deployment in intelligent transportation systems where local decision support is as important as aggregate predictive accuracy.

CHAPTER 6

6: FRONTEND AND VISUALISATION SYSTEM

This chapter presents the frontend and visualisation layer of the proposed FAV-ASTCL traffic forecasting system. While the backend performs the core tasks of data processing, model inference, and API response generation, the frontend is responsible for presenting this information in a clear, interactive, and operationally useful form. In a real-time intelligent transportation system, prediction quality alone is not sufficient; the results must also be delivered through an interface that supports interpretation, monitoring, and rapid decision-making. For this reason, the frontend and visualisation layer is treated as an essential part of the overall system rather than as a secondary implementation detail.

The visualisation system designed in this project aims to bridge the gap between complex spatio-temporal model outputs and user-friendly real-time traffic monitoring. The dashboard is built to display current traffic status, exogenous conditions, and short-term forecasts in an intuitive manner, allowing users to understand both present and anticipated road conditions at a glance. In addition to the primary browser-based dashboard, the system also includes a console streaming mode for lightweight terminal-based monitoring and an alternative Streamlit dashboard for quick demonstration and testing purposes. Together, these interfaces provide flexibility across different usage scenarios.

This chapter begins by describing the UML diagrams used to represent the logical design of the frontend and interaction flow of the system. It then explains the structure and design of the browser dashboard, including its layout, styling system, and JavaScript data flow. Finally, it discusses the sensor-level prediction display, the terminal-based streaming interface, and the Streamlit-based alternative visualisation environment.

6.1 UML Diagrams

To formally represent the structure and interaction behaviour of the frontend and visualisation layer, a set of UML diagrams is included in this project. These diagrams help describe the system from multiple perspectives, including user interaction, component relationships, execution flow, and operational logic. UML diagrams are useful in a project of this nature because they provide a standardised way to communicate how the system behaves without depending only on source code explanations.

6.1.1 Use Case Diagram

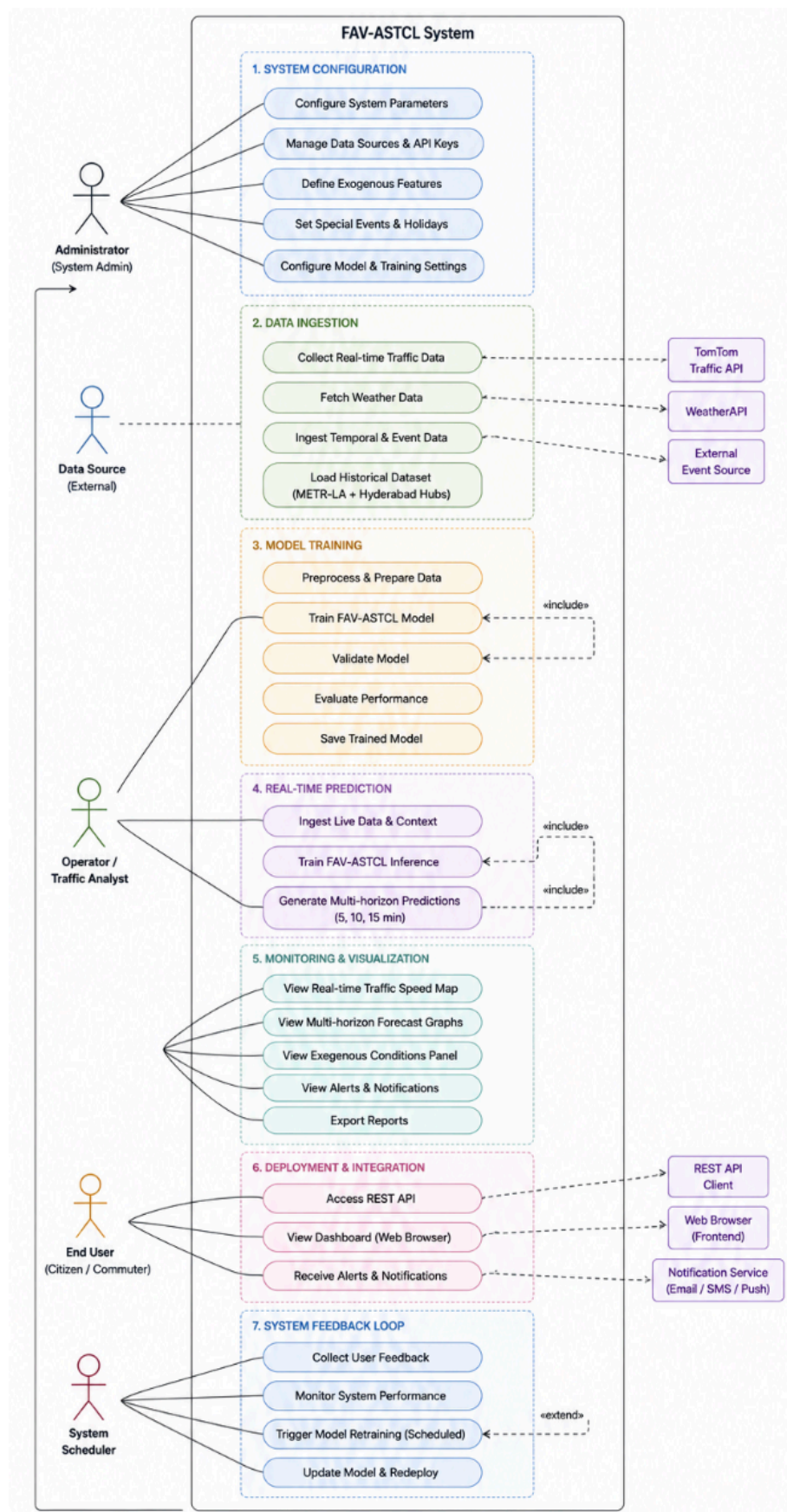


Fig: 6.1 Use Case Diagram

6.1.2 Class Diagram

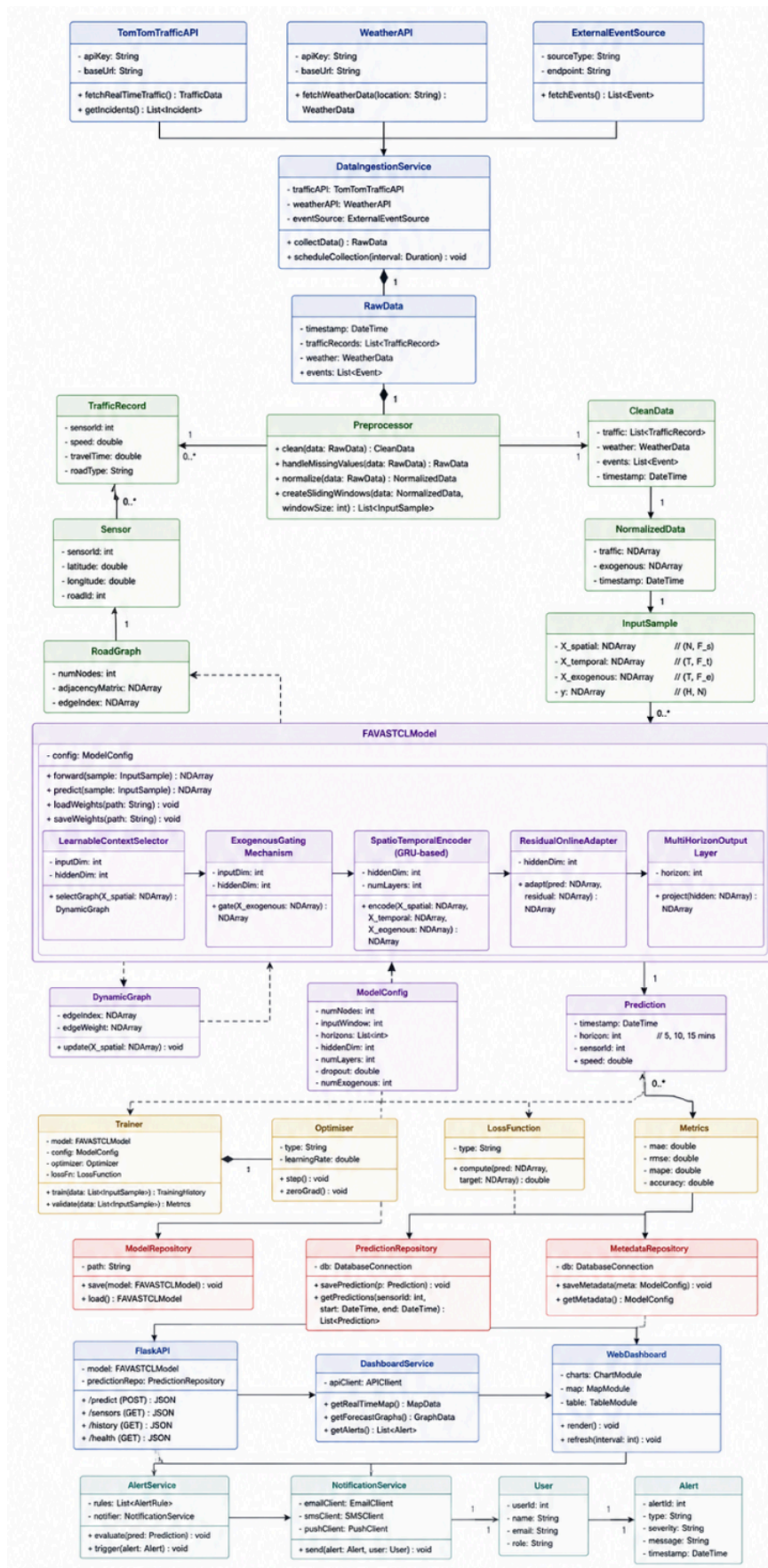


Fig: 6.2 Class Diagram

6.1.3 Sequence Diagram

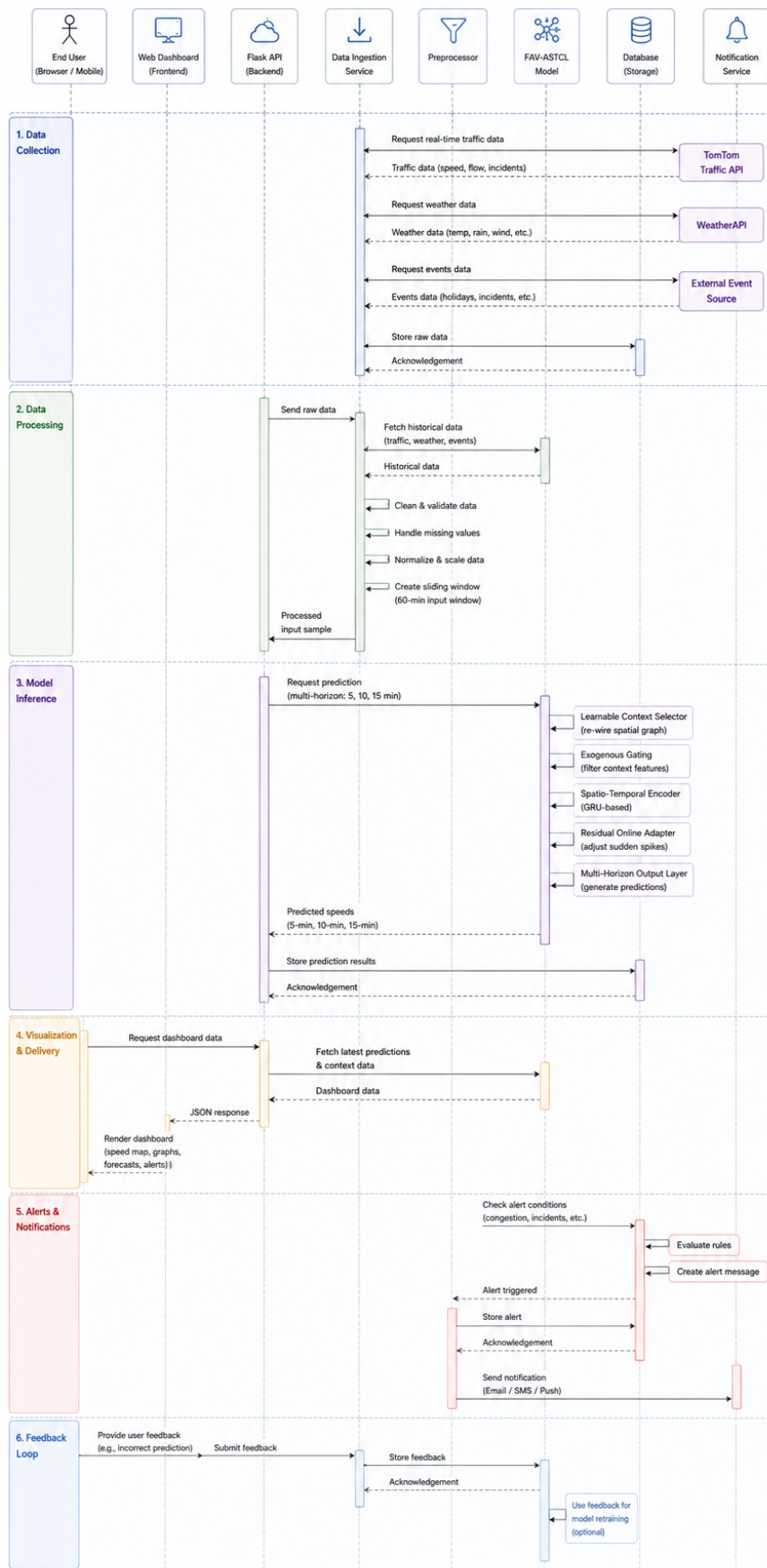


Fig: 6.3 Sequence Diagram

6.1.4 Activity Diagram

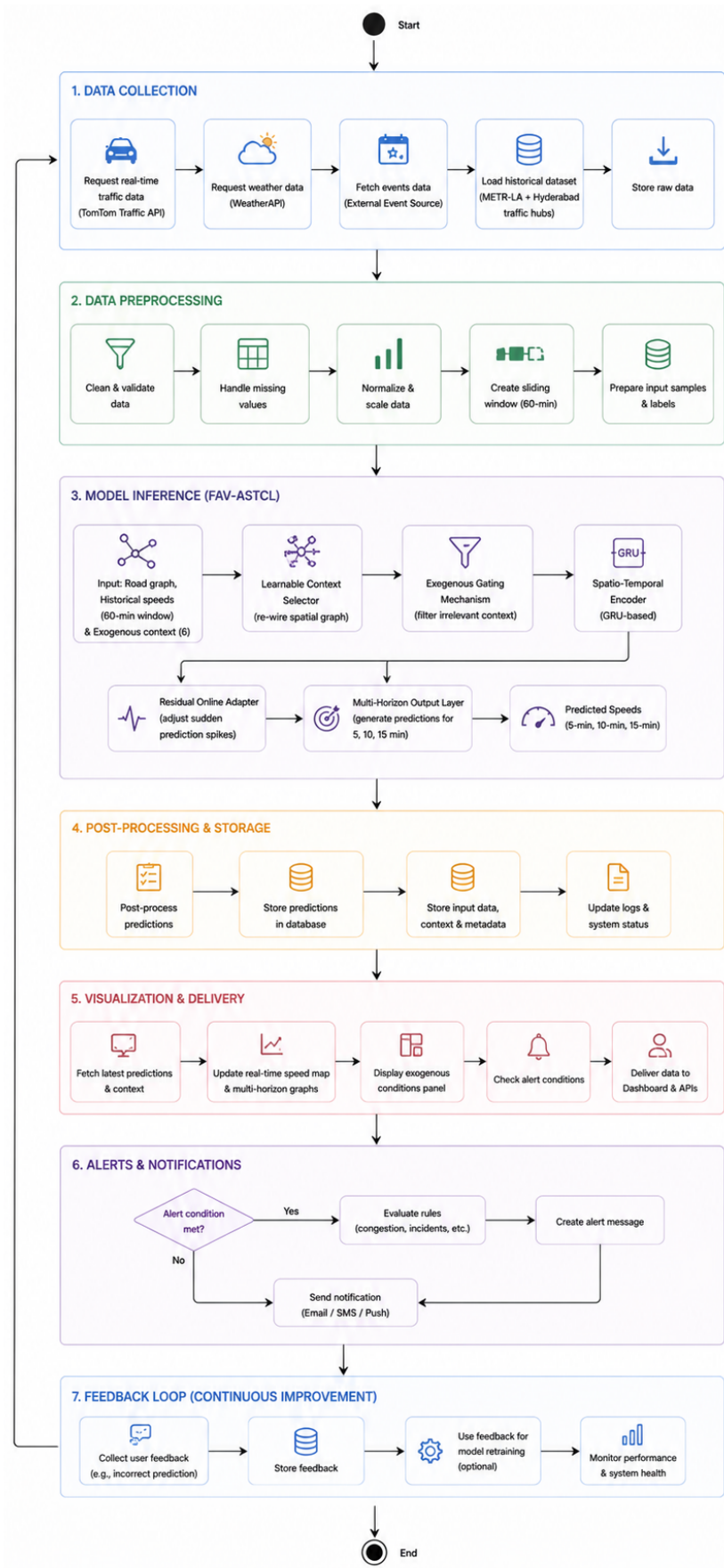


Fig: 6.4 Activity Diagram

6.2 Frontend Dashboard

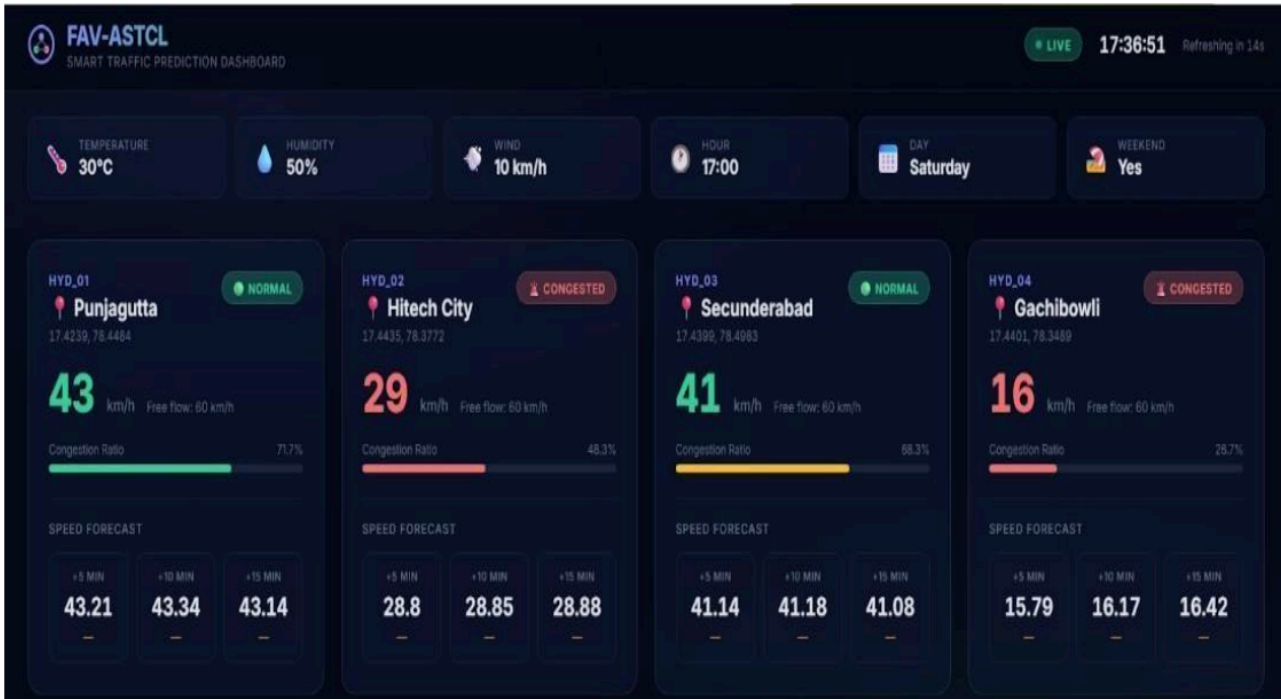


Fig: 6.5 Frontend Dashboard

The browser dashboard is the primary user-facing visualisation interface of the proposed traffic forecasting system. It is designed to present real-time traffic information in a structured, interpretable, and visually coherent manner. Since the backend returns a rich set of values for each location, including current speed, free-flow speed, congestion ratio, and multi-step forecasts, the dashboard must organise this information in a way that avoids clutter while still preserving operational usefulness.

The dashboard is implemented using standard web technologies, namely HTML, CSS, and JavaScript. This choice is deliberate. A lightweight implementation without a heavy frontend framework keeps the system simple, fast, and easier to deploy. It also aligns with the project's broader emphasis on practical engineering and production-oriented design. By avoiding unnecessary dependencies, the dashboard remains highly portable and can be deployed alongside the Flask server without requiring a separate frontend build pipeline.

The visual design follows a modern monitoring-dashboard style with a dark theme, responsive card layout, live status elements, and contextual panels. This design is intended to support both readability and aesthetics. In real-time systems, the quality of visual hierarchy matters greatly because users often need to extract important information very quickly. The dashboard therefore uses clear grouping, semantic colour coding, consistent spacing, and prominent numeric displays to improve interpretability.

6.2.1 Page Layout

```
1 <div class="sensor-card" id="HYD_01">
2   <h3>Punjagutta</h3>
3   <p class="coords">17.4239, 78.4484</p>
4   <div class="speed">42.0 km/h</div>
5   <div class="status normal">NORMAL</div>
6   <div class="forecast-grid">
7     <span>5 min: 44.12</span>
8     <span>10 min: 46.30</span>
9     <span>15 min: 45.87</span>
10  </div>
11 </div>
```

The dashboard layout is divided into multiple functional sections, each with a specific purpose in the information hierarchy. At the top of the page is the header section, which identifies the system and establishes that the interface is operating in live mode. This header may include the project title, a live status badge, a digital clock, and a refresh countdown. These elements immediately inform the user that the data is continuously updated and not merely a static snapshot.

Below the header lies the contextual information section, which displays the current exogenous conditions used by the prediction system. This section may contain values such as temperature, humidity, wind speed, current hour, weekday name, and weekend status. Including this information directly in the dashboard is important because it makes the contextual reasoning of the model visible to the user. Rather than treating exogenous variables as hidden backend inputs, the dashboard surfaces them explicitly, thereby increasing interpretability.

The main body of the page consists of a responsive grid of sensor cards. Each card corresponds to one monitored traffic location and provides a compact summary of the most important predictive information. This includes the sensor identifier, location name, coordinates, current speed, congestion status, congestion ratio bar, and future forecasts for 5, 10, and 15 minutes. By placing these values in a card-based layout, the dashboard allows users to compare multiple locations side by side without excessive scrolling or visual confusion.

The footer section typically displays auxiliary information such as the last updated timestamp or project label. Although visually less prominent, this information contributes to the sense of system freshness and operational trustworthiness. Altogether, the page layout is designed to

guide the eye from general system status to contextual overview and finally to location-specific predictive details.

6.2.2 CSS Design System

The CSS design system defines the visual identity and usability principles of the dashboard. A dark theme is adopted because it is commonly used in monitoring interfaces and provides strong contrast for numeric traffic values, coloured status badges, and highlighted forecast elements. The dark background also helps reduce visual strain during prolonged observation, especially in operational environments such as control rooms or analysis workstations.

```
1  :root {
2    --bg-deep: #050a18;
3    --bg-card: rgba(15, 25, 50, 0.65);
4    --accent: #818cf8;
5    --green: #34d399;
6    --red: #f87171;
7    --radius: 16px;
8  }
9
10 .sensor-card {
11   background: var(--bg-card);
12   border-radius: var(--radius);
13   padding: 1rem;
14   color: white;
15   box-shadow: 0 8px 24px rgba(0, 0, 0, 0.25);
16 }
17
18 .status.normal { color: var(--green); }
19 .status.congested { color: var(--red); }
```

The design system relies on reusable visual tokens such as background colours, text colours, accent colours, border radii, card shadows, and spacing values. This ensures consistency across all parts of the interface. For example, green may be used to represent normal traffic flow, amber may indicate moderate congestion, and red may indicate severe congestion. By applying these semantic colours consistently, the dashboard becomes more intuitive and easier to interpret quickly.

The card-based structure uses rounded containers, layered backgrounds, and subtle shadow or transparency effects to separate visual regions without introducing clutter. Large numeric

```

1  async function fetchData() {
2      const response = await fetch('/api/predictions');
3      const data = await response.json();
4      renderContext(data);
5      renderSensors(data.sensors);
6  }
7
8  setInterval(fetchData, 30000);
9  fetchData();

```

displays are used for current speed values so that the most important operational variable is immediately visible. Supporting labels and metadata are styled with lighter contrast and smaller text to preserve hierarchy. This distinction between primary and secondary information is important in any dashboard because it prevents the interface from appearing flat or overwhelming.

Responsiveness is another key aspect of the CSS design. The sensor grid is designed to adapt to different screen sizes so that the dashboard can function on desktops, laptops, or smaller devices. Card widths, font sizes, and spacing are adjusted in a way that preserves readability even when the layout changes. This makes the visualisation system more flexible and increases its practical usability.

6.2.3 JavaScript Data Flow

The JavaScript layer acts as the dynamic engine of the dashboard. While HTML provides the page structure and CSS defines the visual presentation, JavaScript is responsible for fetching live data from the backend and updating the dashboard contents at regular intervals. In this project, the JavaScript logic is intentionally kept modular and focused so that the data flow remains easy to understand and maintain.

```

1  function renderSensors(sensors) {
2      sensors.forEach(sensor => {
3          const card = document.getElementById(sensor.sensor_id);
4          card.querySelector('.speed').textContent = `${sensor.current_speed} km/h`;
5          card.querySelector('.status').textContent = sensor.congestion_status;
6      });
7  }

```

The process begins with an asynchronous API call to the backend prediction endpoint. Once the response is received, the JavaScript code parses the JSON object and distributes the data to different visual components. The context panel is updated with the latest environmental and temporal values, while the sensor grid is updated with the most recent traffic and forecast information. This update mechanism is repeated automatically at a fixed refresh interval, usually every 30 seconds.

A well-designed JavaScript flow ensures that elements are updated in place rather than recreated from scratch during each refresh cycle. This helps reduce flicker and provides a smoother visual experience. It also allows animations or transition effects to be applied when values change, such as highlighting an updated speed or refreshing a congestion bar. Such small interactions improve usability by making it obvious that the dashboard is actively receiving live data.

The JavaScript layer also handles conditional rendering logic. For example, congestion status labels may change colour based on threshold values, trend indicators may point upward or downward depending on predicted traffic direction, and error handling logic may display fallback states if a data request fails. In this way, JavaScript not only moves data into the page but also gives the dashboard behavioural intelligence.

6.3 Sensor-Level Prediction View

One of the most important features of the dashboard is the sensor-level prediction view. This view transforms the model's raw outputs into a compact operational panel for each monitored traffic location. Since traffic forecasting is most meaningful when tied to specific road regions, the sensor-level display plays a central role in making the system practically useful.

Each sensor card includes a combination of present-state information and future-state prediction. The present-state information typically consists of the current traffic speed and free-flow speed, which together help the user understand how close the road segment is to ideal travel conditions. The congestion ratio derived from these values provides a convenient summary of traffic severity, while the congestion status label converts that ratio into a more immediately interpretable category such as normal or congested.

The future-state component of the card displays the multi-step forecasts generated by the FAV-ASTCL model. Instead of showing only a single predicted value, the dashboard includes forecasts for 5, 10, and 15 minutes ahead. This gives the user a sense of direction rather than just an isolated estimate. If forecasted values decrease steadily, the user can infer worsening traffic conditions. If they rise gradually, the user can infer recovery. Presenting the predictions in this



Fig: 6.6 Sensor-Level Prediction View

structured multi-horizon format makes the dashboard far more useful for short-term monitoring and planning.

Another strength of the sensor-level prediction view is its ability to support quick comparison across locations. Since all cards follow the same layout and visual language, the user can easily scan across multiple traffic hubs and identify which areas are worsening, which are stable, and which are improving. This is one of the main advantages of using card-based visualisation for multi-location forecasting systems.

6.4 Console Streaming Mode

In addition to the browser dashboard, the system includes a console streaming mode. This mode is designed for lightweight monitoring directly from the terminal and is especially useful in backend-focused environments where a graphical interface may not be necessary or available. Although simpler than the browser dashboard, it still plays an important role in testing, debugging, and operational observation.

In console streaming mode, the system runs in a continuous loop and prints updated traffic information at fixed time intervals. Each cycle typically includes the current timestamp,

environmental context, sensor identifiers, current traffic values, congestion status, and the three forecasted speeds. The output is arranged in a readable textual form so that a developer or operator can quickly inspect the state of the system without opening a browser.

This mode is particularly helpful during development and deployment because it allows the backend prediction pipeline to be verified independently of the frontend. If the model is producing incorrect values or the API integrations fail, these problems can often be identified quickly through terminal output. For this reason, the console mode serves not only as an alternative interface but also as an engineering support tool.

Another benefit of this mode is its low resource usage. Since it does not require browser rendering or frontend assets, it can run easily on remote servers, cloud instances, or headless environments. This makes it suitable for backend diagnostics and simple operational logging.

6.5 Streamlit Dashboard (Alternative UI)

As an alternative to the main HTML-CSS-JavaScript dashboard, the project also includes a Streamlit-based visualisation interface. This interface is intended primarily for rapid prototyping, demonstration, and experimentation. Streamlit provides a convenient way to build data-driven interfaces in Python without manually managing frontend code, which makes it especially useful in academic and development settings.

```
1 import streamlit as st
2
3 st.title("FAV-ASTCL Traffic Dashboard")
4 st.metric("Current Speed", "42.0 km/h")
5 st.metric("5-Min Forecast", "44.12 km/h")
6 st.metric("Congestion Status", "NORMAL")
```

The Streamlit dashboard displays the same core information as the browser dashboard, including sensor-level traffic status, current contextual variables, and multi-step speed predictions. However, its design is simpler and more framework-driven compared to the custom browser implementation. The advantage of this approach is speed of development. Since Streamlit is tightly integrated with Python, it allows the model and visualisation logic to be connected with minimal overhead.

The Streamlit version is particularly useful for quick testing because it can be launched with a single command and does not require maintaining separate frontend scripts. It also supports features such as automatic refresh and lightweight caching of model resources, which are helpful for interactive demonstrations. At the same time, it is less flexible than the custom dashboard in terms of fine-grained control over design, animation, and frontend behaviour.

For this reason, the Streamlit dashboard is best viewed as a complementary interface rather than a replacement for the primary browser dashboard. It demonstrates the adaptability of the system and shows that the forecasting backend can support multiple presentation layers depending on the deployment requirement. In a research setting, this flexibility is valuable because it allows the same backend model to be reused across different user interfaces with minimal modification.

CHAPTER 7

7. FUTURE SCOPE

This chapter outlines the future directions through which the proposed FAV-ASTCL system can be extended from a research prototype into a more comprehensive, city-scale intelligent transportation platform. Although the present work demonstrates strong forecasting performance and a complete end-to-end deployment pipeline, it also opens several opportunities for deeper technical improvement and broader practical adoption. Traffic forecasting is not a static problem; it exists within a rapidly changing urban ecosystem influenced by infrastructure growth, real-time sensing, edge computing, cloud analytics, and smart mobility services. For this reason, the future scope of the project must be viewed not merely as optional enhancement, but as the natural next stage in the evolution of the system.

The current implementation already establishes a strong foundation by combining adaptive spatio-temporal learning, exogenous feature integration, and deployable visualisation tools. However, a fully mature traffic intelligence platform would require richer real-time data acquisition, more advanced deep learning techniques, larger-scale urban connectivity, stronger system interoperability, and tighter integration with public mobility infrastructure. Future work can therefore proceed in multiple directions simultaneously: improving forecasting intelligence, improving deployment realism, and improving decision support capabilities.

The sections that follow discuss these opportunities in detail. The scope begins with real-time smart city deployment, then moves toward model-level enhancements, integration with IoT and cloud systems, expansion into mobile and web ecosystems, and finally the broader role of the framework within intelligent traffic management systems.

7.1 Real-Time Smart City Deployment

One of the most important future directions for this work is full real-time deployment within a smart city environment. In its current form, the system demonstrates how live traffic and weather data can be integrated with a forecasting model and exposed through an operational dashboard. However, a real smart city deployment would go far beyond a small set of monitored nodes and would require the model to function across a dense, continuously updating urban sensing network.

A real-time deployment architecture would involve persistent ingestion of traffic data from a wide range of urban sources such as road-side sensors, GPS-based fleet tracking systems, CCTV analytics pipelines, public transit feeds, toll plaza records, and connected vehicle telemetry. Instead of querying a small number of traffic points periodically, the system could ingest continuous

streams from hundreds or thousands of road segments across the city. This would make the forecasting model much more representative of actual metropolitan traffic conditions and would allow the system to support city-scale spatio-temporal awareness rather than isolated corridor-level monitoring.

In a smart city setting, the backend would need to operate with rolling temporal buffers for each location instead of approximating history from repeated current readings. This is a critical improvement because real deployment depends heavily on genuine temporal continuity.

Maintaining live sequence histories per sensor would allow the model to exploit its temporal design properly and would produce forecasts that more closely reflect the dynamics of congestion growth, dissipation, spillover, and route diversion. Such a deployment would also support more reliable adaptation during rush hours, accidents, weather shifts, and special-event traffic surges.

Another key direction in smart city deployment is geographic scalability. The system can be expanded from selected urban traffic hubs to a full city-wide topology involving arterial roads, intersections, ring roads, business corridors, and suburban connectors. This would require the development of a more comprehensive spatial graph, potentially constructed from actual road maps, transportation planning data, and sensor placement strategies. As the network grows, the forecasting framework would become more useful not only for road users but also for traffic control centers, municipal authorities, and urban planners.

Real-time deployment would also benefit from location-specific retraining or transfer learning. Although the current work demonstrates strong predictive performance, the long-term value of the model would be significantly improved by training on city-native traffic histories rather than relying mainly on benchmark transfer. Hyderabad-specific or city-specific sensor archives would allow the network to learn the true behavioural patterns of local road infrastructure, driving styles, congestion timing, and seasonal effects. Over time, this would transform the system from a generic forecasting framework into a genuinely localised urban intelligence engine.

In addition, future deployment could incorporate regional zoning. Instead of treating the entire city as a uniform graph, the system could partition the network into commercial zones, industrial corridors, transit-oriented regions, residential areas, and event-sensitive hotspots. Such zoning could improve both interpretability and control because predictions would then be linked more closely to the functional role of each urban region. This type of region-aware forecasting could be highly valuable in smart governance applications where traffic management decisions depend on spatial priorities rather than purely numerical outputs.

7.2 Advanced Model Enhancements

Although FAV-ASTCL already improves substantially on the baseline architecture, there remains considerable scope for advancing the predictive model itself. Traffic systems are highly nonlinear, dynamic, and context-sensitive, which means there is no single final architecture that solves the forecasting problem completely. Future research can therefore explore more expressive ways of modeling spatial dependencies, temporal uncertainty, multimodal context, and city-scale adaptation.

The current model introduces adaptive graph learning, exogenous gating, and residual correction, all of which strengthen forecasting robustness. However, these mechanisms can be expanded further by incorporating more advanced graph construction strategies, richer attention mechanisms, hierarchical temporal modeling, and uncertainty-aware learning. These improvements would allow the architecture to capture increasingly subtle traffic behaviours, particularly under highly disrupted or non-recurring conditions.

Two especially promising directions are improved dynamic graph learning and the adoption of more advanced deep learning architectures. These are discussed separately below.

7.2.1 Improved Dynamic Graph Learning

The Learnable Context Selector in the current system generates an adaptive graph from the current traffic context, which already represents a major improvement over purely static adjacency approaches. However, future work can make this mechanism more sophisticated in several important ways. One possible enhancement is the use of multi-scale dynamic graph learning, where spatial relationships are learned separately for local, regional, and city-wide interaction patterns. In practical traffic systems, nearby roads often influence each other immediately, while distant roads may become related only through larger congestion propagation patterns. A multi-scale graph design would allow the model to capture both short-range and long-range traffic dependencies more effectively.

Another promising enhancement is edge-conditioned or feature-aware graph construction. Instead of deriving dynamic connectivity only from traffic states, the graph learning mechanism could also consider external factors such as weather severity, event density, road category, lane count, or time-dependent route relevance. This would produce a more semantically meaningful graph where the relation between two nodes depends not just on observed speed similarity but also

on contextual traffic conditions. For example, two areas might become strongly linked only during office commuting hours, festival periods, or rainfall conditions. Encoding such conditional

connectivity would make the model more adaptive and behaviourally realistic.

Temporal graph evolution is another important research direction. In the present architecture, a dynamic graph is generated for each inference step, but future systems could explicitly model how the graph itself evolves over time. This would allow the model to learn not just the current spatial dependencies, but also the trajectory by which those dependencies change. In practical terms, such a mechanism could help capture how congestion spreads from one corridor to another over a series of intervals, rather than treating spatial reconfiguration as an instantaneous independent event.

A further improvement could involve hybrid graph construction, where learned dynamic relationships are combined with map-based structural priors, traffic rules, and mobility flow statistics. This would create a richer spatial representation that balances physical infrastructure knowledge with data-driven adaptation. Such a hybrid graph could be especially useful in dense urban settings where traffic influence is shaped not only by road proximity but also by signal timing, turning movements, one-way constraints, and behavioural rerouting.

Future work may also explore sparsity-aware graph learning. As city-scale sensor networks grow larger, fully dense learned adjacency matrices may become computationally expensive and less interpretable. Sparse adaptive graphs, where only the most important dynamic relations are retained, could improve scalability while preserving the most meaningful interactions. This would make the framework more suitable for very large urban deployments with hundreds or thousands of nodes.

Overall, improved dynamic graph learning represents one of the strongest future avenues for extending the model. Since traffic forecasting is fundamentally a spatial reasoning problem, better graph adaptation directly translates to better understanding of how congestion patterns emerge, propagate, and dissolve across complex road systems.

7.2.2 Advanced Deep Learning Architectures

Beyond graph learning improvements, the model can also be enhanced through more advanced neural architectures. The current framework uses GRU-based temporal modeling, which is efficient and effective for sequential prediction. However, future work could investigate more expressive temporal modules such as transformer-based encoders, temporal attention networks, long-range memory architectures, or hybrid recurrent-attention systems.

Transformer architectures are particularly promising because they can model long-range dependencies more flexibly than recurrent networks. In traffic systems, current congestion may sometimes be influenced by patterns that extend beyond the immediate short history window, especially during sustained rush periods, public events, or recurring network disturbances.

Attention-based architectures could help the model selectively focus on the most relevant historical moments rather than compressing all sequence information into a single recurrent hidden state. This may improve performance when the temporal relevance of past events is irregular or delayed.

Another future enhancement is the use of spatio-temporal attention at multiple levels. Instead of applying graph operations and temporal modeling as separate stages, future models could jointly attend across nodes, time steps, and exogenous signals. This would allow the system to learn where, when, and under what conditions traffic dependencies matter most. Such architectures could be especially useful in complex urban networks where the importance of context shifts rapidly across both space and time.

Uncertainty-aware deep learning is another important frontier. The current model produces point forecasts, but future systems could generate prediction intervals, confidence estimates, or probabilistic traffic scenarios. This would make the system more useful in operational decision-making because traffic managers often need to understand forecast reliability in addition to forecast value. For example, a prediction of severe congestion with low uncertainty is operationally more actionable than a similar prediction with high uncertainty. Incorporating Bayesian techniques, quantile forecasting, ensemble learning, or probabilistic decoders could make the system significantly more informative.

Multi-task learning also offers strong future potential. Instead of forecasting only traffic speed, the model could be extended to jointly predict congestion level, travel time, anomaly likelihood, and incident probability. Such multi-output learning may improve overall representation quality because related transportation tasks often share useful hidden structure. A model that simultaneously understands speed evolution and congestion intensity may develop richer internal representations than one trained on a single signal alone.

Another possible direction is continual learning or online adaptation. Urban traffic systems evolve over time due to construction, route changes, policy interventions, and behavioural shifts. A future version of the model could adapt incrementally as new data arrives, rather than requiring full retraining at fixed intervals. This would make the forecasting system more resilient to long-term drift and better suited to always-on smart city deployment.

These advanced deep learning directions would not merely increase model complexity for its own sake. Their real value lies in improving generalisation, interpretability, uncertainty awareness, and long-term usability under complex real-world traffic conditions.

7.3 IoT and Cloud Integration

A major future direction for this project is deeper integration with Internet of Things infrastructure and cloud-based computing ecosystems. Smart transportation systems increasingly depend on distributed sensing and scalable backend services, which means that a forecasting model must eventually operate within a broader network of data-producing and data-consuming components. Moving in this direction would transform the current implementation from a contained prediction service into a fully connected intelligent mobility platform.

On the IoT side, the system could be linked directly with road-side devices such as loop detectors, smart cameras, radar sensors, connected traffic lights, pollution monitors, and vehicle telemetry units. These devices can provide richer and more granular real-time observations than periodic API polling alone. Integrating them would improve both the frequency and diversity of the input data, allowing the model to respond more rapidly to changing road conditions. In addition, IoT edge devices could perform initial preprocessing or filtering near the source, reducing latency and bandwidth consumption before the data is sent to the central inference system.

Cloud integration would enable scalable storage, processing, retraining, and service orchestration. Large-scale urban traffic systems generate continuous high-volume streams, which require reliable infrastructure for ingestion, time-series storage, model serving, logging, alerting, and analytics. Cloud-native deployment would allow the forecasting backend to run in a more resilient and elastic environment, where services can scale up during peak traffic hours and scale down during off-peak periods. This would be especially important if the platform expands to monitor hundreds of sensors across multiple urban regions.

Another significant benefit of cloud integration is support for data lakes and historical analytics. By continuously storing incoming sensor, weather, and forecast data in a structured cloud repository, the system could support retrospective analysis, seasonal pattern discovery, anomaly mining, and long-term policy evaluation. These capabilities would extend the usefulness of the project beyond short-term forecasting into strategic urban planning and transportation research.

A cloud-integrated architecture would also make it easier to support multiple interfaces simultaneously. Browser dashboards, mobile applications, public APIs, municipal monitoring

platforms, and third-party routing services could all consume the same cloud-hosted prediction services. This would improve interoperability and make the system more adaptable to diverse stakeholders.

In the future, one can also imagine a hybrid edge-cloud architecture. In such a design, lightweight inference or anomaly detection would occur at the edge near the sensor network, while more complex forecasting, retraining, and global optimisation tasks would be handled in the cloud. This would provide a balance between low latency and high computational capability, making the system more robust for real-time traffic control.

7.4 Mobile and Web Platform Expansion

Another valuable future direction is the expansion of the system into broader mobile and web platforms. At present, the project includes a browser-based dashboard and an alternative Streamlit interface, both of which are suitable for demonstration and monitoring. However, a full deployment ecosystem would benefit from platform-specific interfaces designed for different user groups such as traffic authorities, commuters, emergency response teams, logistics operators, and urban planners.

A dedicated mobile application could provide real-time traffic prediction access to users on the move. For commuters, such an application could display short-term speed forecasts, congestion alerts, route advisories, and estimated road recovery times. For traffic officers or field operators, the same application could provide location-based monitoring panels, incident-sensitive updates, and mobile decision support tools. Because mobile devices are often the most accessible platform for real-time information delivery, this direction could significantly increase the practical reach of the system.

A richer web platform could also be developed beyond the current dashboard. Instead of only displaying sensor cards and forecasts, the expanded web version could include map-based overlays, trend charts, historical playback, comparative time-window analysis, exportable reports, and role-specific access levels. Such a platform could allow users to zoom into particular traffic zones, compare current conditions with previous days, and visualise the spatial spread of congestion. This would make the system far more powerful for both operational monitoring and analytical review.

Platform expansion also creates opportunities for user personalisation. Different users need different levels of detail. A commuter may need simple travel guidance, while a city traffic analyst may require dense network-level views and historical trend tools. A flexible web and mobile

ecosystem could support multiple presentation layers over the same backend forecasting engine, thereby increasing utility without duplicating prediction logic.

Another promising direction is public information integration. The forecasting outputs could be embedded into public city portals, transit websites, or commuter assistance services. For instance, predicted congestion warnings for major corridors could be shared through civic dashboards, digital signage, or multimodal journey planners. This would move the project from a specialist research system toward a publicly beneficial mobility service.

In future versions, geospatial interfaces could also be introduced. Instead of only card-based summaries, users could view traffic predictions on interactive road maps with color-coded segments, sensor markers, and time-slider based forecast exploration. Such interfaces are often far more intuitive for large-scale spatial interpretation and would greatly enhance the visual analytics capability of the platform.

7.5 Intelligent Traffic Management Systems

The most impactful long-term future scope of this work lies in its integration with intelligent traffic management systems. In its current form, the proposed model predicts short-term traffic speeds and presents them through operational dashboards. However, future systems can move beyond prediction alone and become active components of traffic control, planning, and decision-making.

An intelligent traffic management system would use the forecasting outputs not just to observe the future, but to influence it. Predicted congestion patterns could be used to support adaptive traffic signal timing, lane management, route diversion recommendations, public transport prioritisation, emergency vehicle corridor preparation, and incident response planning. In this setting, the forecasting model becomes part of a closed-loop decision system in which predictions inform control actions, and the outcomes of those actions are then observed and learned from over time.

For example, if the model predicts worsening congestion at a major intersection over the next 15 minutes, a traffic control center could use that information to adjust signal phases in advance rather than reacting only after the congestion becomes severe. Similarly, route guidance systems could reroute certain vehicle flows proactively, reducing the formation of downstream bottlenecks. Such predictive control represents a major advancement over reactive traffic management and is one of the strongest practical justifications for intelligent forecasting systems.

Another future direction is integration with multimodal urban mobility management. Traffic conditions are closely linked with buses, metro feeder routes, parking systems, ride-sharing services, freight movement, and pedestrian flows. A mature intelligent traffic management platform could therefore use traffic forecasts alongside transit load data and event information to coordinate multiple aspects of city mobility. This would make the system more valuable not just for road traffic operations, but for broader transportation ecosystem optimisation.

The framework could also support alert generation and prioritised intervention. If the system detects a likely congestion surge near hospitals, transport terminals, school zones, or emergency routes, it could automatically notify relevant authorities or trigger heightened monitoring. In this way, forecasting becomes part of a proactive urban safety and resilience mechanism rather than just a predictive analytics exercise.

Over the longer term, the system could contribute to digital twin environments for smart cities. In such environments, the forecasting model would operate as one component within a virtual representation of the city, continuously ingesting data, simulating likely traffic futures, and testing intervention strategies before they are applied in the physical world. This would greatly increase the strategic value of the project and align it with emerging smart city governance paradigms.

Ultimately, integration with intelligent traffic management systems is where the full societal value of this work can be realised. The proposed FAV-ASTCL model has already shown that accurate short-term forecasting is feasible with adaptive spatio-temporal learning. The next step is to connect this predictive ability with action-oriented traffic governance so that urban mobility systems become not only observable, but truly intelligent, anticipatory, and responsive.

CHAPTER 8

8. CONCLUSION

The project began with the recognition that short-term traffic forecasting remains a difficult but highly important problem in modern urban mobility systems. Traffic patterns are influenced by complex and time-varying interactions across road networks, and these interactions are further affected by contextual factors such as weather, time, and local activity conditions. Traditional forecasting systems that rely only on temporal history or static spatial relationships often struggle to adapt to rapidly changing traffic conditions. The present work addressed this limitation by proposing a forecasting framework that combines adaptive spatial learning, exogenous context integration, temporal sequence modeling, and a deployment-ready visualisation system. The final outcome is therefore not only a predictive model, but a complete intelligent traffic forecasting pipeline that includes model design, backend serving, frontend presentation, and practical real-time integration.

8.1 Project Summary

The project focused on the design and implementation of FAV-ASTCL, an enhanced spatio-temporal traffic forecasting system developed to improve short-term speed prediction under dynamic urban conditions. The model was built on top of the ASTCL baseline and extended through three key architectural ideas: a Learnable Context Selector for adaptive graph formation, an Exogenous Gating Mechanism for selective integration of contextual variables, and a Residual Online Adapter for correcting abrupt temporal changes during inference. These additions were motivated by the observation that traffic networks are not static in behaviour and that a forecasting model must be able to adapt its spatial and contextual reasoning in real time.

Beyond model development, the project also implemented a complete backend service using Python, PyTorch, and Flask, along with live external data integration through traffic and weather APIs. This backend was connected to a custom frontend dashboard, a terminal-based streaming interface, and an alternative Streamlit UI, making the system demonstrable as an end-to-end deployment framework rather than only an offline research model. A project summary should convey the project's purpose, the steps taken to achieve it, and the resulting outcomes; this work does so through a full pipeline from forecasting architecture to user-facing deployment. As a result, the project contributes both at the algorithmic level and at the systems-engineering level.

The work also demonstrated that traffic forecasting can be presented in an interpretable operational format. Instead of ending at benchmark evaluation alone, the project showed how

prediction outputs can be transformed into sensor-level traffic insights, congestion indicators, and multi-horizon visual summaries. This strengthens the practical value of the work and shows that the proposed framework is not only academically meaningful but also suitable for future adaptation into real smart-city applications.

8.2 Objectives Achieved

The primary objective of the project was to improve the predictive capability of a spatio-temporal traffic forecasting system by addressing three limitations commonly seen in earlier models: dependence on static graph structures, weak handling of exogenous contextual information, and limited responsiveness to abrupt traffic fluctuations. This objective was achieved through the successful design and implementation of FAV-ASTCL, in which each of these issues was addressed through a dedicated architectural component. The adaptive graph mechanism improved spatial flexibility, the exogenous encoder and gating mechanism strengthened contextual learning, and the residual correction module improved responsiveness to short-term disturbances.

A second objective was to evaluate the model on a recognised benchmark dataset and demonstrate measurable performance improvement over the ASTCL baseline. This objective was also achieved. The model was trained and tested under a standard multi-horizon forecasting setup using the METR-LA dataset, which is a commonly used benchmark for traffic forecasting studies. Earlier chapters reported strong gains over ASTCL in terms of MAE, RMSE, and MAPE, indicating that the proposed system substantially improves both average and relative predictive accuracy on the benchmark task.

A third objective was to move beyond offline experimentation and create a deployable system capable of real-time inference and visualisation. This objective was fulfilled through the implementation of a Flask-based backend, integration with external traffic and weather APIs, and development of multiple user interfaces including a browser dashboard, console view, and Streamlit dashboard. This deployment-oriented design is significant because many research projects stop at the stage of model evaluation, whereas this work demonstrates how the predictive model can be connected to live data sources and presented in a practical decision-support format.

A further objective was to maintain interpretability and modularity in the system design. This was achieved through clear separation of concerns across the architecture, including standalone modules for model components, API logic, data integration, and visual presentation. Such modularity improves maintainability and reflects established software engineering principles

for structured frontend and backend design. Taken together, the project successfully met its technical, analytical, and deployment-oriented goals.

8.3 Key Contributions

One of the central contributions of this project is the proposal of FAV-ASTCL as a more adaptive and context-aware extension of ASTCL for short-term traffic forecasting. The Learnable Context Selector enables the network to infer dynamic inter-sensor relationships from current traffic behaviour rather than relying only on a predefined static graph. This contribution is important because traffic influence patterns often shift across time and road conditions, and a fixed adjacency representation cannot always capture such changes adequately. By allowing the graph structure to adapt during inference, the system becomes more responsive to non-stationary traffic conditions.

A second major contribution is the integration of exogenous variables through a dedicated encoder and gating mechanism. In many traffic prediction systems, contextual information such as weather and time is either omitted or fused too simplistically. In contrast, this project treats exogenous context as a selective influence rather than as a passive appended feature set. The gating mechanism allows the model to regulate the extent to which contextual variables contribute to the prediction process, thereby improving robustness and reducing the risk of irrelevant context dominating the forecast.

A third contribution is the Residual Online Adapter, which helps correct the smoothed temporal representation generated by the recurrent encoder. Traffic systems often exhibit abrupt transitions caused by congestion surges, incidents, or sudden recovery, and such transitions can be difficult to model using sequence encoders alone. The residual correction mechanism improves the model's ability to respond to these changes without compromising the broader temporal representation. This addition strengthens the practical usefulness of the system in volatile traffic conditions.

Another important contribution lies in the engineering implementation of the project. The work does not stop at theoretical architecture design or benchmark performance comparison. It extends into backend deployment, REST API serving, sensor-level forecast generation, and multi-interface visualisation. This broad implementation scope is a significant contribution because it transforms the work into a complete intelligent transportation prototype rather than a purely analytical exercise.

Finally, the project contributes to the idea of interpretable deployment-oriented traffic AI. Through the dashboard, console stream, and alternative UI paths, the forecasting results are presented in

forms that can be understood and acted upon by users. This strengthens the connection between deep learning research and operational transportation systems, which is one of the most important long-term goals in applied intelligent mobility research.

8.4 System Performance Overview

The overall performance of the proposed system indicates that FAV-ASTCL is both technically effective and practically viable. In benchmark evaluation, the model achieved a MAE of 3.633 km/h, an RMSE of 8.842, and a MAPE of 8.22%, outperforming the ASTCL baseline, which recorded a MAE of 9.034 km/h, an RMSE of 16.173, and a MAPE of 23.47%. These improvements show that the proposed model reduces average prediction deviation, reduces sensitivity to large forecasting errors, and improves relative predictive accuracy across varying traffic conditions. The resulting overall prediction accuracy was reported as 91.78%, compared to 76.53% for the baseline. These earlier reported results support the conclusion that the proposed architecture delivers a meaningful improvement in forecasting quality within the chosen experimental setting.

The system also performed well from an implementation perspective. The model was successfully integrated into a backend service capable of responding to live requests, combining contextual information, generating forecasts, and returning structured JSON outputs for frontend use. The browser dashboard, console streaming mode, and Streamlit dashboard demonstrated that the model's outputs can be translated into usable operational views. This indicates that the project is not only accurate at the modeling level but also coherent at the systems level, where data ingestion, inference, and user interaction must work together reliably.

Another aspect of performance worth noting is the balance between complexity and practicality. Although the architecture introduces multiple advanced components, it remains modular and computationally manageable enough to support deployment-oriented experimentation. This is particularly valuable in applied machine learning systems, where very high modeling complexity can sometimes limit usability. The current project shows that it is possible to achieve strong predictive gains while still preserving clarity of design and deployability of the overall pipeline.

At the same time, the performance overview must be interpreted realistically. Technical report guidance emphasises that conclusions should consolidate findings and implications while remaining grounded in what was actually demonstrated. The reported results establish strong evidence that the proposed architecture improves benchmark traffic forecasting and supports

operational visualisation, while also leaving room for future enhancement through city-specific retraining, richer live data buffering, and larger-scale deployment.

8.5 Final Remarks

The completion of this project demonstrates that effective short-term traffic forecasting requires more than the use of a powerful neural network alone. It requires the ability to adapt spatial relationships dynamically, to incorporate external context intelligently, to preserve temporal sensitivity, and to present results in a form that supports understanding and action. FAV-ASTCL addresses these requirements by combining adaptive graph learning, context-aware fusion, residual temporal refinement, and deployment-ready system design within a single unified framework. From a research perspective, the project shows that extending a baseline architecture with carefully targeted mechanisms can produce substantial improvements in predictive performance. From an engineering perspective, it shows that such a model can be wrapped into a practical system that communicates with live services, serves predictions through an API, and visualises results through multiple interfaces. A strong conclusion should reinforce the importance of the work, summarise the main findings, and indicate why the results matter beyond the immediate project context. In that regard, the present work demonstrates both academic value and practical relevance.

In final terms, this project establishes a strong foundation for future intelligent transportation research and deployment. It confirms that adaptive, context-aware spatio-temporal learning can improve traffic forecasting performance and that such intelligence can be embedded into a functional real-time monitoring system. As cities continue to move toward data-driven and predictive mobility management, systems of this kind will become increasingly important. The present work therefore stands not only as a completed project, but as a meaningful step toward smarter, more responsive, and more deployable traffic intelligence solutions.

REFERENCES

1. W. Huang, G. Song, H. Hong, and K. Xie, “Deep architecture for traffic flow prediction: Deep belief networks with multitask learning,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 15, no. 5, pp. 2191–2201, 2014.
2. S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
3. K. Cho, B. Van Merriënboer, Ç. Gülçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” *arXiv preprint arXiv:1406.1078*, 2014.
4. M. Defferrard, X. Bresson, and P. Vandergheynst, “Convolutional neural networks on graphs with fast localized spectral filtering,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
5. Y. Li, Y. Rose, C. Shahabi, and Y. Liu, “Diffusion convolutional recurrent neural network: Data-driven traffic forecasting,” in *International Conference on Learning Representations (ICLR)*, 2018.
6. B. Yu, H. Yin, and Z. Zhu, “Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting,” in *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)*, 2018.
7. Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, “Graph WaveNet for deep spatial-temporal graph modeling,” in *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.
8. L. Bai, L. Yao, C. Li, X. Wang, and C. Wang, “Adaptive graph convolutional recurrent network for traffic forecasting,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
9. C. Tian and W.-K. Chan, “Spatial temporal multi-graph convolutional network for urban traffic flow prediction,” *IET Intelligent Transport Systems*, vol. 15, no. 8, pp. 1021–1031, 2021.
10. C. Zheng, X. Fan, C. Wang, and J. Qi, “GMAN: A graph multi-attention network for traffic prediction,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
11. Z. Lin, J. Feng, Z. Lu, Y. Li, and D. Jin, “DeepSTN+: Context-aware spatial-temporal neural network for crowd flow prediction in metropolis,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, pp. 1020–1027, 2019.

12. C. Shang, J. Chen, and J. Bi, “Discrete graph structure learning for forecasting multiple time series,” in International Conference on Learning Representations (ICLR), 2021.
13. T. Kipf, E. Fetaya, K.-C. Wang, M. Welling, and R. Zemel, “Neural relational inference for interacting systems,” in International Conference on Machine Learning (ICML), 2018.
14. K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
15. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), 2017.